Generative Models

“The brain has about 10^{14} synapses and we only live for 10^{9} seconds. So we have a lot more parameters than data. This motivates the idea that we must do a lot of unsupervised learning since the perceptual input (including proprioception) is the only place we can get 10^{5} dimensions of constraint per second.”

-Geoffrey Hinton (2014 AMA on Reddit)

proprioception – perception or awareness of the position of the body

A Theoretical View of Unsupervised Learning

- A way to capture patterns in raw data in a label-free way
  - Finding all patterns equivalent to short description of raw data (low Kolmogorov complexity)
    - Length of the shortest computer program that produces the output
  - Shortest code-length equivalent to optimal inference (Solomonoff induction)
    - This is a theory of prediction based on logical observations. The underlying assumption is that the environment follows some unknown but computable probability distribution
    - Not computable but approximations such as AIXI have been developed
      - This is a reinforcement learning agent which interacts with some stochastic but unknown environment. The only goal is to maximize the sum of rewards over *m* time steps
    - Another approximation are *Inductive Turing Machines*

Solomonoff Induction

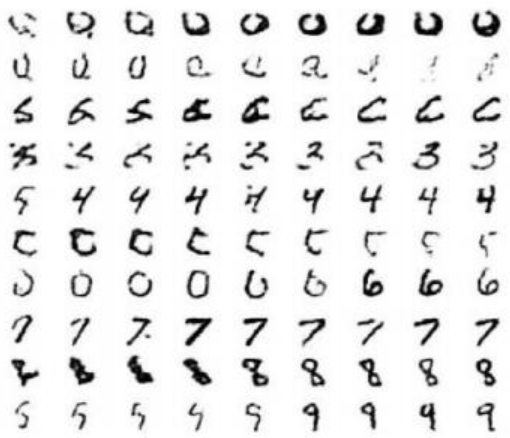
- Occam’s Razor: (Wikipedia) Occam's razor says that when presented with competing hypotheses that make the same predictions, one should select the solution with the fewest assumptions,
- Solomonoff Induction explains observations of the world by the smallest computer program that outputs those observations. Solomonoff proved that this explanation is the most likely one, by assuming the world is generated by an unknown computer program. That is to say the probability distribution of all computer programs that output the observations favors the shortest one.

AIXI

- Modeled as *reinforcement learning* for agent interacting with an *unknown* but *computable* environment μ
- The game goes from $t = 1$ to N
- At each time step, the agent chooses an action a_t and the environment returns a reward r_t and observation o_t distributed according to the conditional probability
- This conditional distribution is not known to the agent. However, it is *computable* by a Turing machine $\mu(o_t r_t | a_1 o_1 r_1 \ldots a_{t-1} o_{t-1} r_{t-1} a_t)$
- The goal of the agent is to maximize the sum of rewards
- The action chosen at time step t is based on maximizing the sum of reward from t to N using a stochastic policy. It uses a program q running on a Turing machine to compute this
  - Imagine the agent continuously refines q over time
- Note: In reinforcement learning, past history is ignored (Markovian) but not here

Applications of Unsupervised Learning

- Find new patterns in data
  - Useful in developing new medicines
- Generate new data
  - For example, “draw a cat” or “write a book”
- Predict the next element of a sequence
  - Autoregression
- Compression
  - Find the most compact representation – can be lossy or lossless



[Deep Belief Nets, Hinton, Osindero, Teh, 2006]

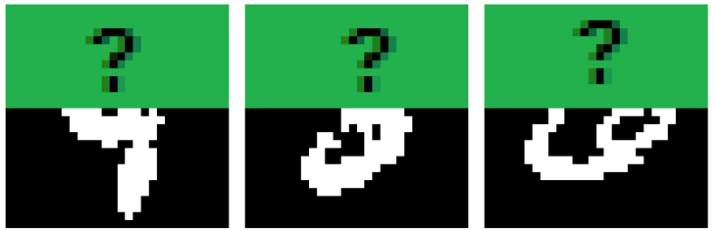
[GAN, Goodfellow et al. 2014]

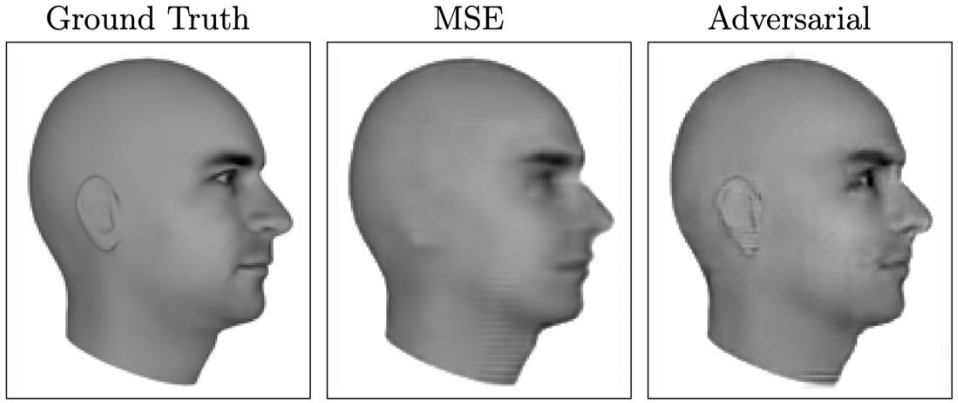
[DCGAN, Radford, Metz, Chintala 2015]

[CycleGAN: Zhu, Park, Isola & Efros, 2017]

Why Study Generative Models?

- Can be used to infer missing data
- Enable ML to work with multi-modal outputs





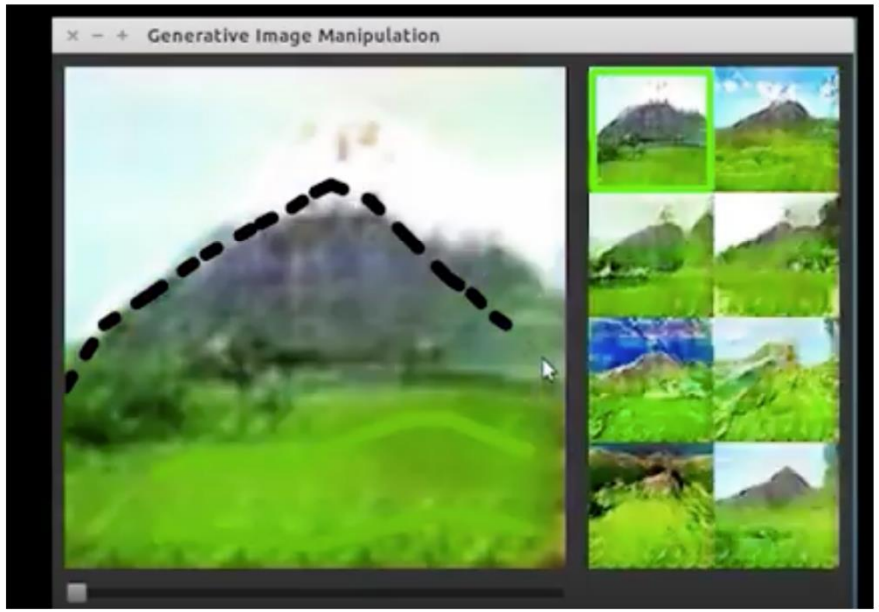
(Lotter et al 2015)

A model is trained to predict the next frame in a video sequence. The first image is the actual image that the model should predict. Image in center is predicted when using MSE. There are several possible futures in the video each with slight variations in head positions (multi-modal outputs). MSE picks an “average” resulting in the blurriness. The one on the right is predicted by a GAN

Ledig t al 2016

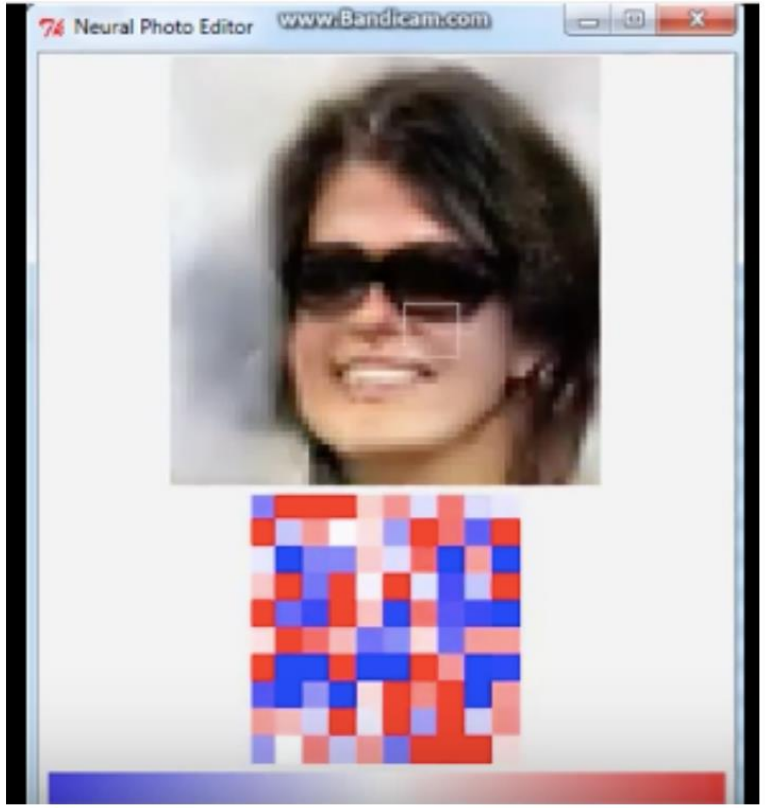
- Single image super-resolution (also multi-modal outputs)
  - The original image is down sampled to make a low-resolution image. Each method then attempts to recreate the original
  - Bicubic – interpolation between neighboring pixels
  - SRResNet using MSE while the one on the right is a GAN based method

- Create art or help edit



User scribbles some lines in green and iGAN converts it into a meadow. User then draws a rough shape (black dashes) and iGAN adds in a mountain. And so on.

Zhu et al 2016



User wants to touch up image. User uses a black brush to paint a portion of hair roughly. The iGAN converts that to a photorealistic image.

Brock et al 2016

How would you generate MNIST like digits?

- Lets assume each pixel is 1/0 only
- 28x28 images means 784 pixels
- Estimate the joint probability of these pixels taking values 1 or 0
- Generate images using this probability distribution
- Problem:
  - The state space is 2^{784} (i.e., there are this many possible 28x28 images)
  - Need to estimate the probabilities of each of these but only have 60k or 2^{16} images

Alternatively: Exploit Structure Using Conditional Distribution

- Consider MNIST: there is a dependence in pixel values of neighboring pixels. Can we use that information?
- Idea: Estimate conditional probabilities of pixel values based on other pixel values
- Say with probability $p(x_1)$ pick value of first pixel to be 1
- Next, use $p(x_2|x_1)$ to pick 0 or 1 for pixel 2, and so on
- Why will this work (in theory)?
  - Because the conditional probabilities are based on the training examples of actual digits

Example

- Say we only have 2 variables x_1 and x_2
- $p(x_1)$ is a histogram
- $p(x_2|x_1)$ is computed using a NN
  - Input is x_1
  - Output is a distribution over x_2
- Thus, we obtain the model $p(x_1, x_2) = p(x_1)p(x_2|x_1)$
- This generalizes to *d* variables since, in general,

$$\log p(x) = \sum_{i=1}^{d} \log p(x_i|x_1, \dots x_{i-1})$$

Is this a Good Model?

- It extends to *d* dimensions requiring *O(d)* parameters instead of *O(exp(d))*
  - For MNIST $d = 784$
- BUT, limited generalization
  - No information is shared among conditionals
  - Still very large amount of data required for estimation
- For example, $p(x_{784} \mid x_1, \ldots, x_{783})$ requires 2^{783} values to be estimated

Parameterized Model

- Define a Bernoulli random variable whose mean is a learned function $f_i : [0,1]^{i-1} \rightarrow [0, 1]$. In other words, the mean of this r.v. is based on the values of $x_1, \ldots, x_{i-1}$
- This function is learned and is represented by a set of parameters θ_i
- Therefore, the total number of parameters that need to be learned are now, $\sum_1^n |\theta_i|$
- Drawbacks,
  - Cannot represent all possible distributions
- Consider methods to estimate θ

One Approach: Likelihood-based Generative Models

- Samples $x^{(1)}, \ldots, x^{(n)}$ taken from p_{data}
- Let the function approximation for this distribution be parameterized by θ. Therefore, we need to estimate $p_\theta \approx p_{data}$

$$\arg\min_{\theta} loss(\theta, x^1, \ldots., x^n)$$

- Need the loss function to:
  - Work with large datasets (millions)
  - Yield θ such we get a good estimate of the distribution
  - Maximum likelihood works well,

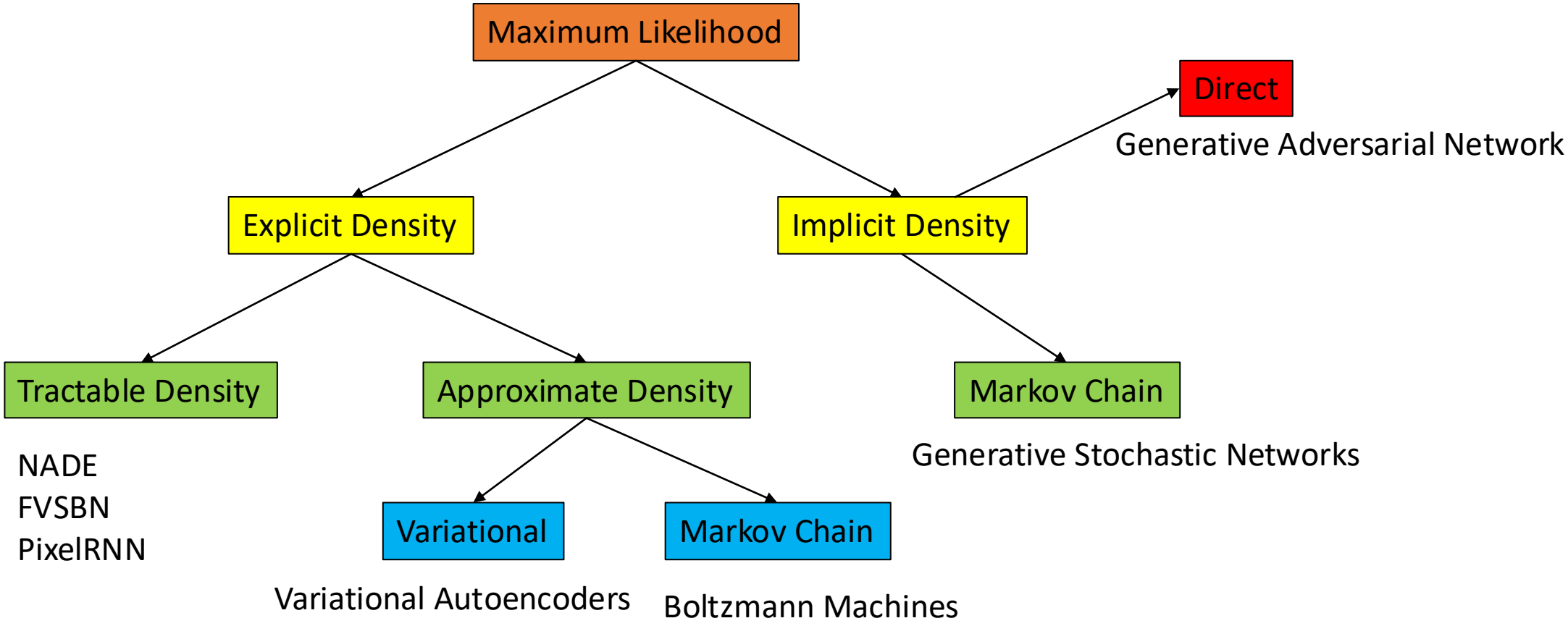
$$\frac{1}{n}\sum_{i=1}^{n} -\log p_\theta(x^i)$$

- Use Stochastic Gradient Descent (SGD) to solve this optimization problem

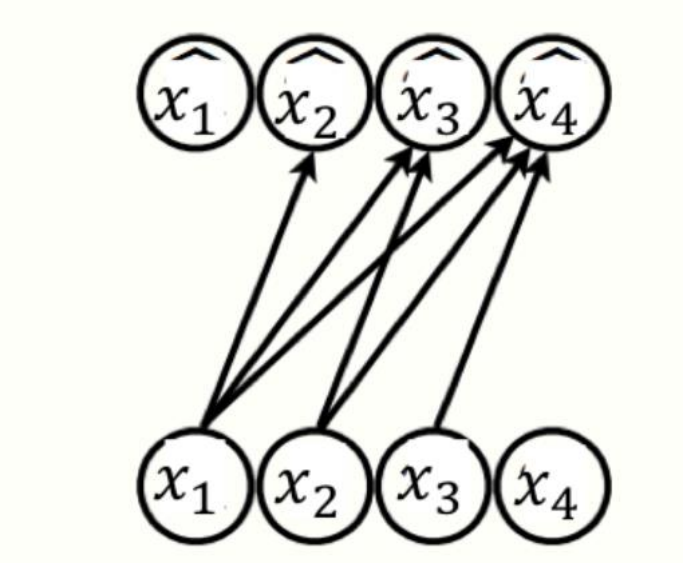
$$\arg\min_{\theta} E_{\{x \sim p_{data}\}}\left[-\log p_\theta(x)\right]$$

Note that we are approximating p_{data} (which is a joint probability distribution over the space of variables) via p_θ

Taxonomy


Maximum Likelihood
Direct
Generative Adversarial Network
Explicit Density
Implicit Density
Tractable Density
Approximate Density
Markov Chain
Generative Stochastic Networks
NADE
FVSBN
PixelRNN
Variational
Markov Chain
Variational Autoencoders
Boltzmann Machines

Fully Visible Sigmoid Belief Network (FVSBN)



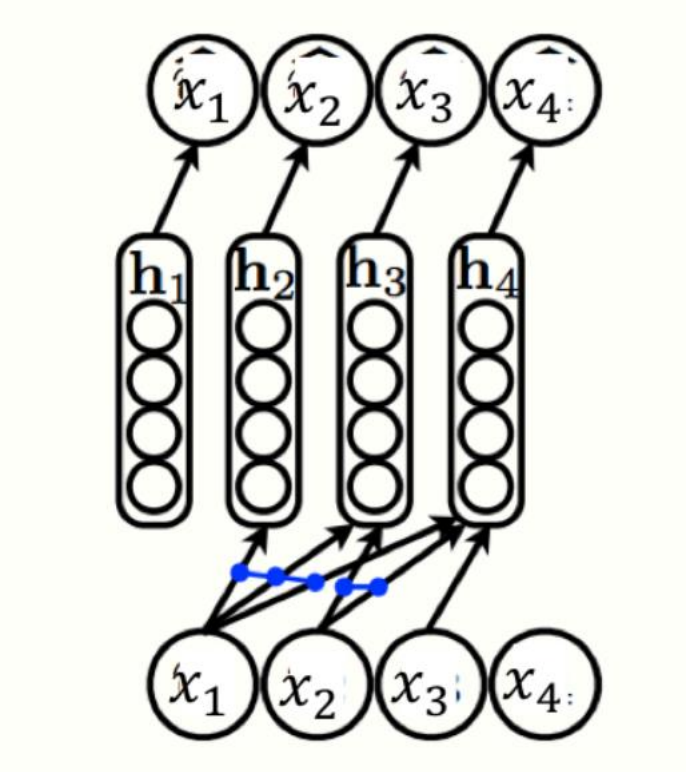
Sigmoid

$$f_i(x_1, \ldots, x_{i-1}) = \sigma(w_0^i + w_1^i x_1 + \ldots + w_{i-1}^i x_{i-1})$$

$$\theta^i = \{w_0^i, \ldots, w_{i-1}^i\}$$

This model may not be powerful enough

Neural Autoregressive Density Estimator (NADE)



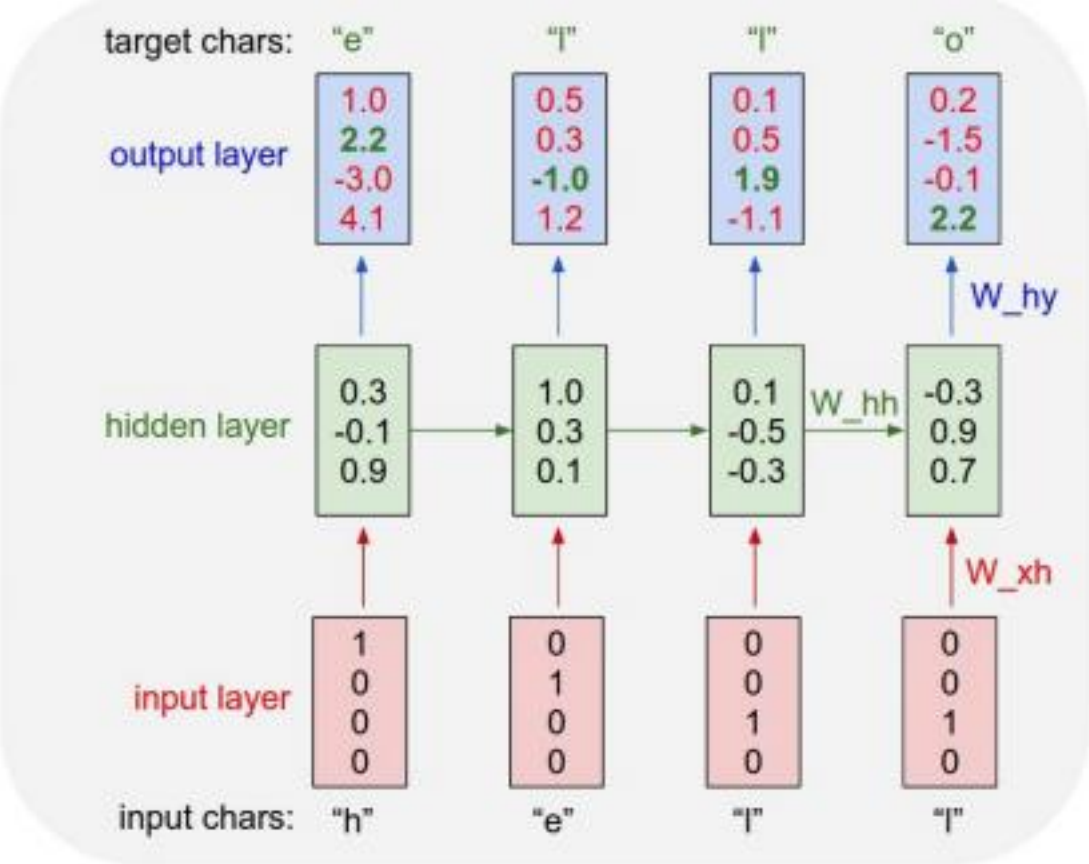
Here, each θ^i is estimated via a NN with a sigmoid activation at the output since we are estimating probabilities. The blue line indicates linked inputs since the same value of x_1 should be used for the other values it affects.

RNN Autoregressive Model

$$\log p(\mathbf{x}) = \sum_{i=1}^{d} \log p(x_i \mid \mathbf{x}_{1:i-1})$$

Sequence of characters

Character at i^{th} position

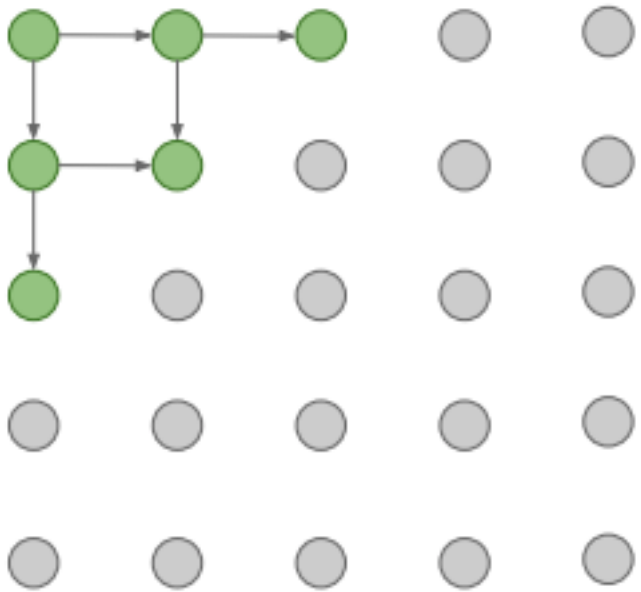


[Karpathy, 2015]

PixelRNN

Generate image pixels starting from corner

Dependency on previous pixels modeled using an RNN (LSTM)



32x32 CIFAR-10

32x32 ImageNet

WaveNet (Google Deepmind)

- Is a FVBN to generate realistic human speech
- Input is text that is broken up into phonemes
- WaveNet uses the sequence of phonemes and already spoken text to generate the next sound – continuous speech and the same "speaker"
- Without text, it generates random sounds

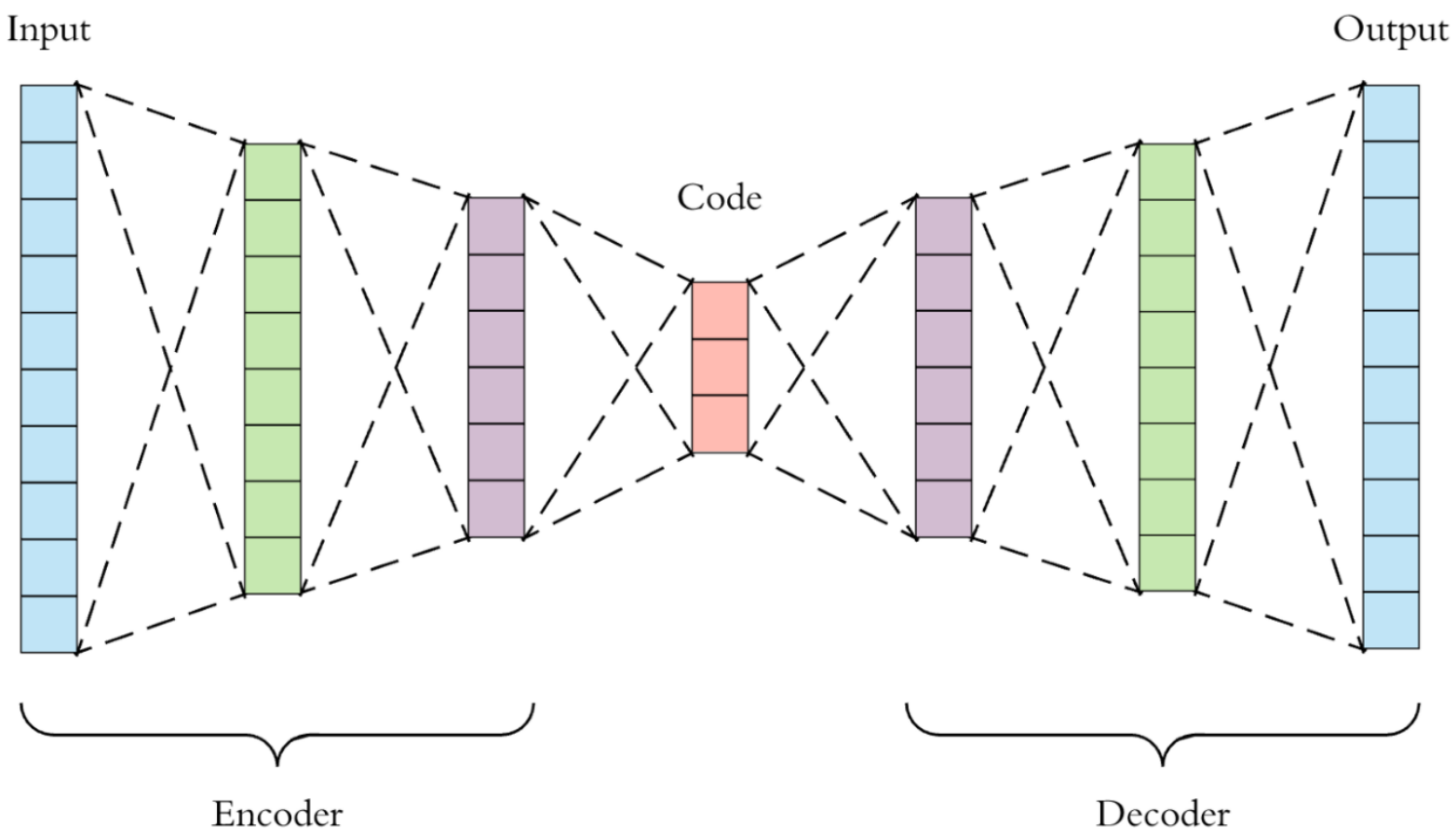
Autoencoders

- An autoencoder attempts to learn h(x) (hidden representation) for input x so we can obtain a reconstruction $\hat{x}$ that is close to x

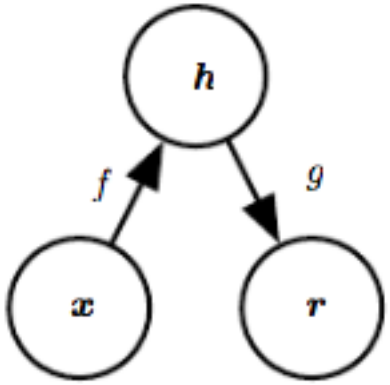
$$h(x) = g(b + Wx)$$
$$\hat{x} = sigmoid(c + Vh)$$

- Note that we constrain the learning so that the output is not just a copy of the input and the hidden representations learn some important statistical property of the input
  - For instance, if the hidden layer is at least as large as the input, it will reconstruct each input perfectly
  - There are methods to prevent this, including using fewer hidden nodes or forcing a sparsity property by adding a sparsity penalty to the loss function

Structure

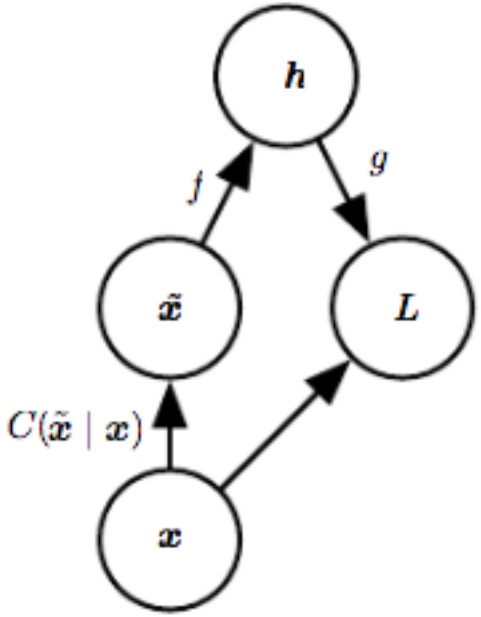


Sparse Autoencoders

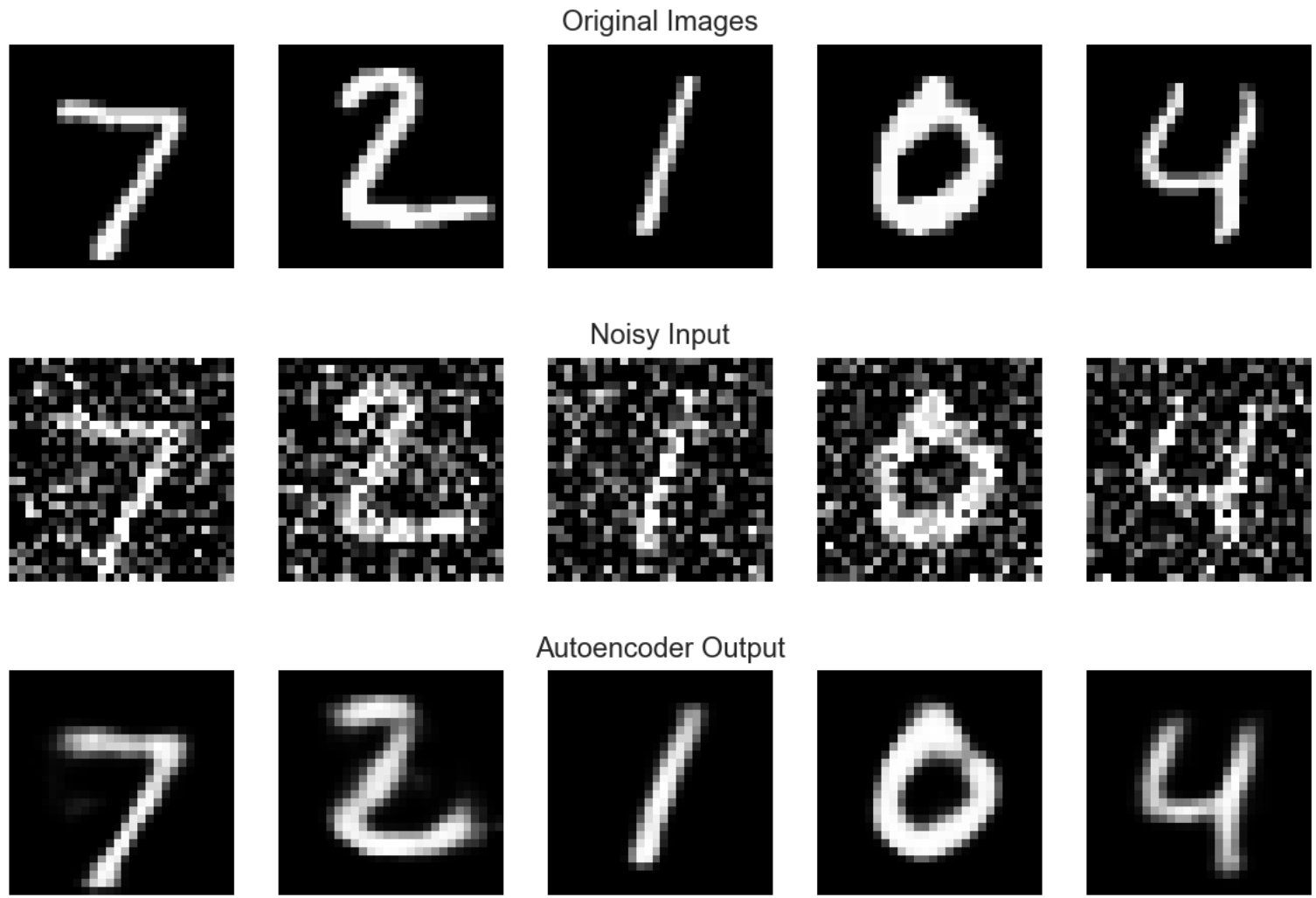


- The learning process can be written as minimizing a loss function $L(x, g(f(x)))$
- An undercomplete autoencoder means *h* has a smaller dimension than *x*
- When the decoder is linear and the loss is the MSE, undercomplete autoencoders learn the same subspace as PCA
- If the encoder and decoder are non-linear and allowed a lot of capacity, they may learn to simply copy the input to the output
- Fix: Use a sparsity penalty $L\left(x, g\left(f(x)\right)\right) + \Omega(\mathrm{h})$ the last term can be thought of a regularizer term
- One choice is $\Omega(\mathrm{h}) = \lambda \sum_i |h_i|$

Denoising Autoencoders



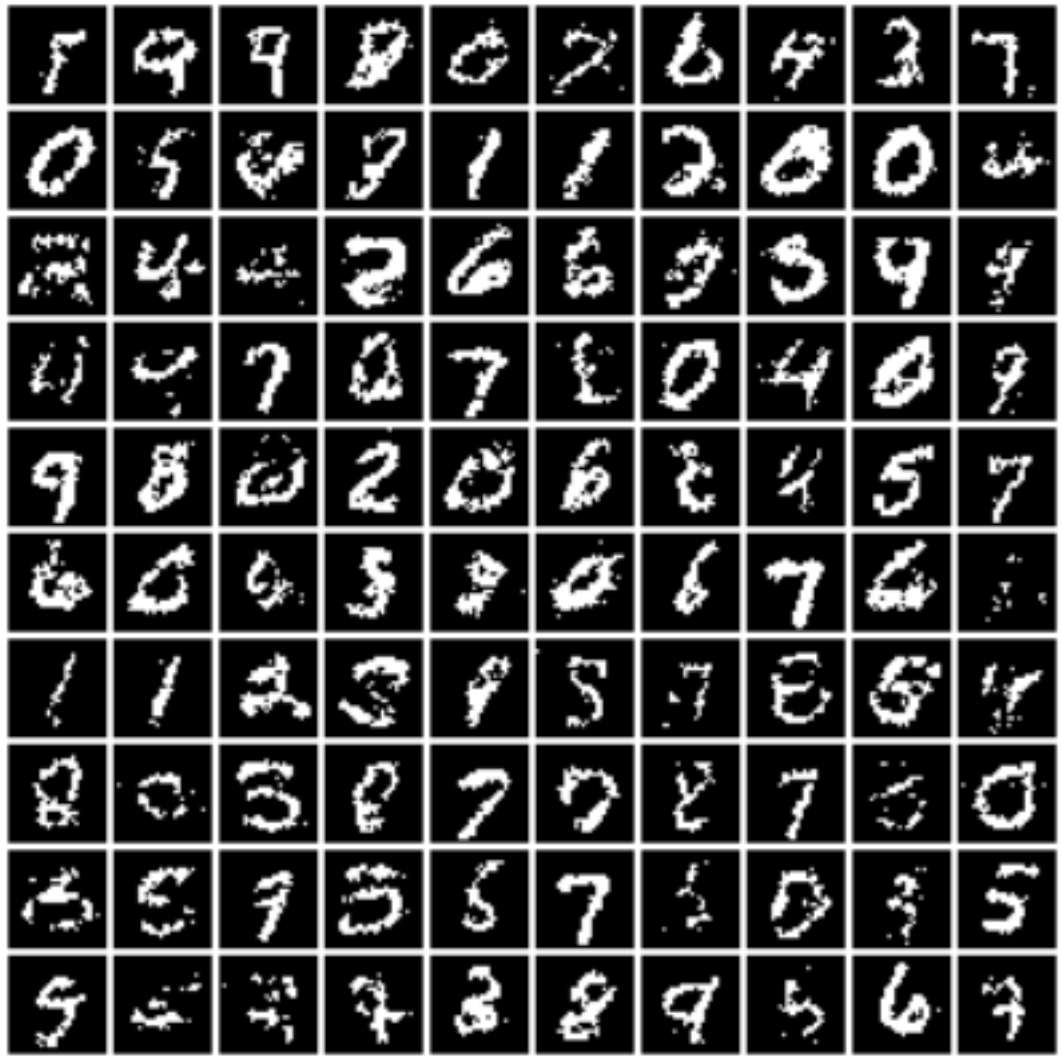
- Sample a training example x
- Sample a corrupted version $\tilde{x}$ from $C(\tilde{x}|x)$
- Learn $p_{reconstruct}(x|\tilde{x}) = p_{decoder}(x|h)$
- Can simply perform gradient descent


Original Images
Noisy Input
Autoencoder Output

Results on MNIST

Used a 0/1 pixel value only for training and output

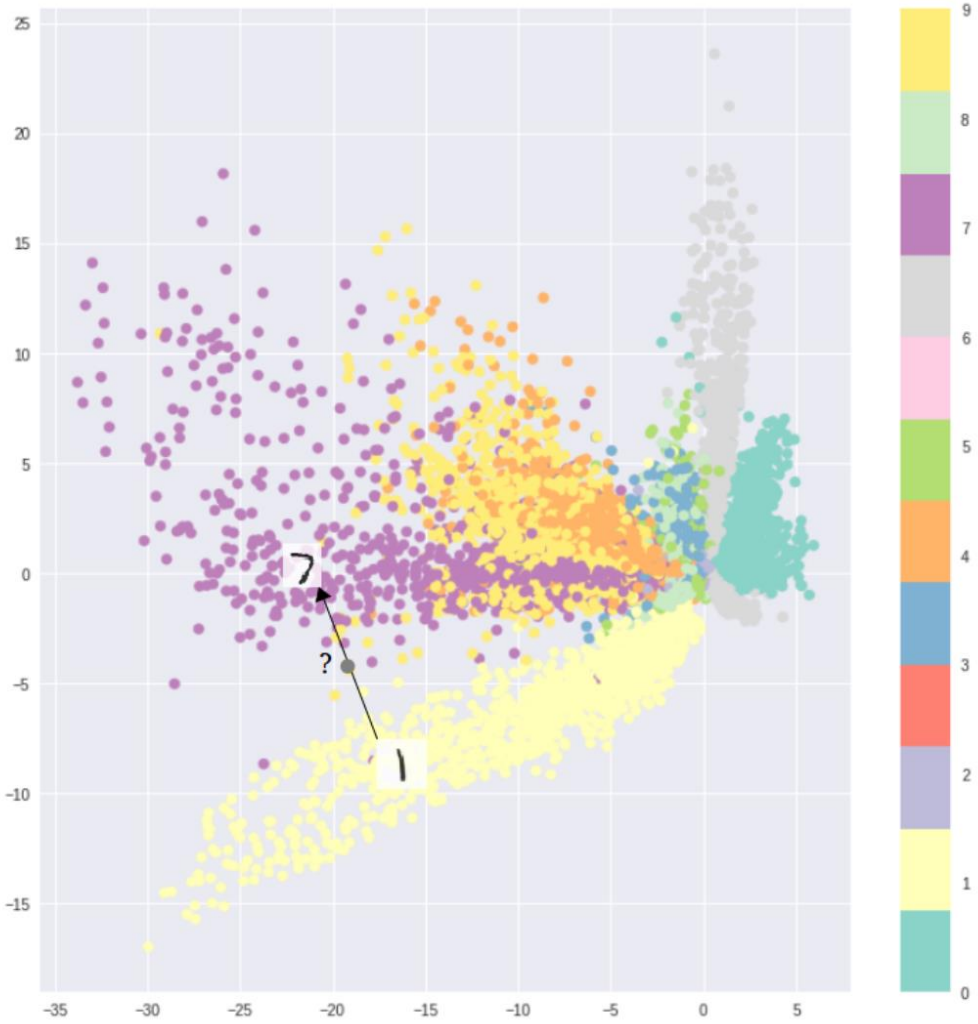
Each row uses a different mask



Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. 2015. MADE: masked autoencoder for distribution estimation. In Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37 (ICML'15), Francis Bach and David Blei (Eds.), Vol. 37. JMLR.org 881-889.

Problem with Using Autoencoders

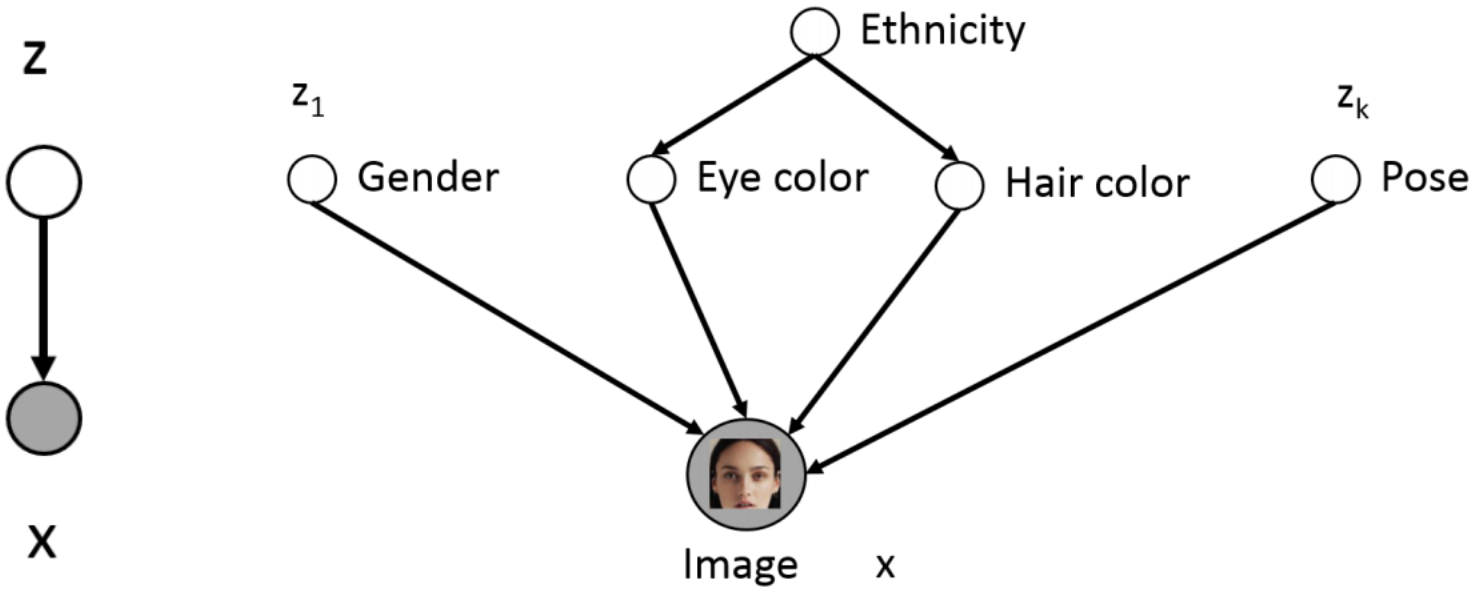
- Standard autoencoders learn a compact representation well
- However, the *latent space* they learn is discontinuous and therefore cannot be used to interpolate to generate new images
  - The autoencoder output is a sampling of a discrete space
- You would like to sample points in between to create novel images. Therefore need AE to learn a continuous space



Optimizing purely for reconstruction loss

What is a Latent Variable?

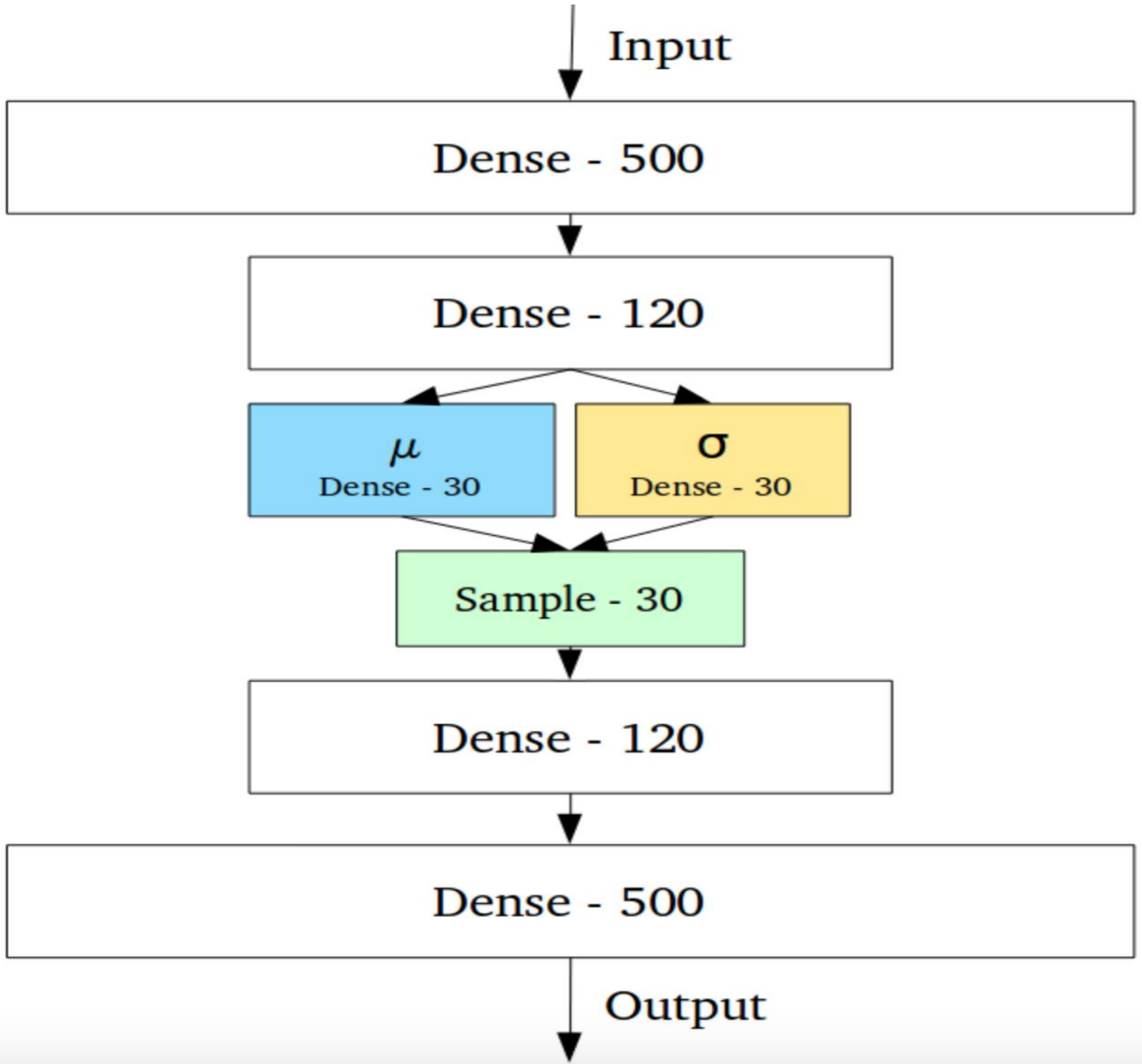
- Lots of variability in images (input *X*) such as hair color, gender, ethnicity. But unless you label them using these attributes for training, these factors of variation are not explicitly available hence they are *latent*



- Only variables *X* are visible (pixel values)
- Latent variables *Z* correspond to high level features
- When you train an autoencoder on images, you minimize the loss between the reconstruction and the original. Hence the latent variables are not visible
- Therefore, asking the AE to produce an image of a “male Asian with green eyes ” is not possible. The AE will produce images that are like the ones it saw. It cannot mix and match these high-level features

Variational Autoencoder (VAE)

The VAE learns the mean and variance of the latent variables. The assumption is that the distribution is a Normal distribution.



Loss Function

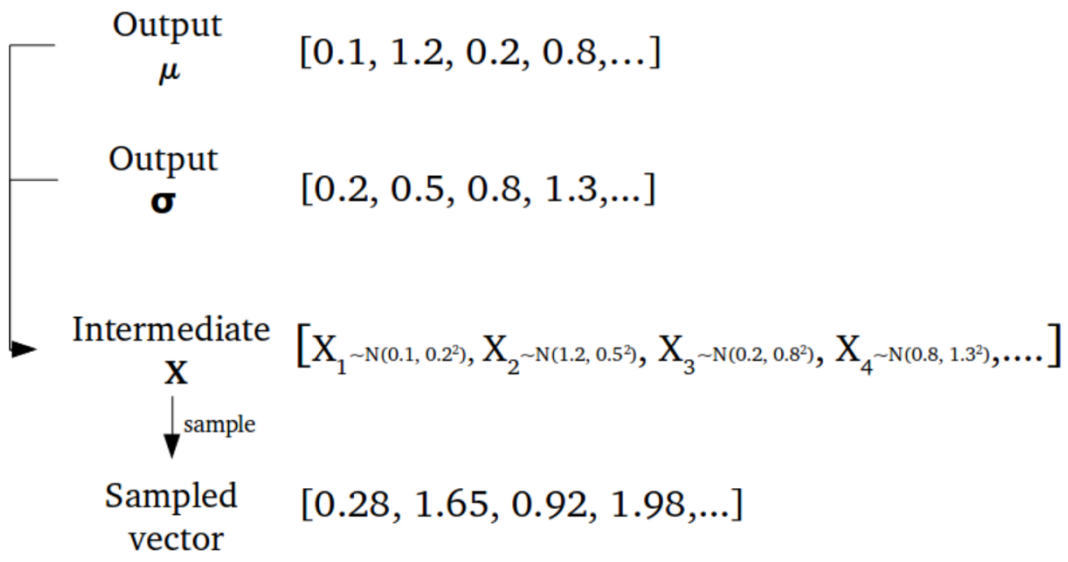
$$\sum_{i=1}^{N} l_i$$

Is the loss computed over *N* data points

$$l_i(\theta, \phi) = -E_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i|z)] + KL(q_\theta(z|x_i)||p(z))$$

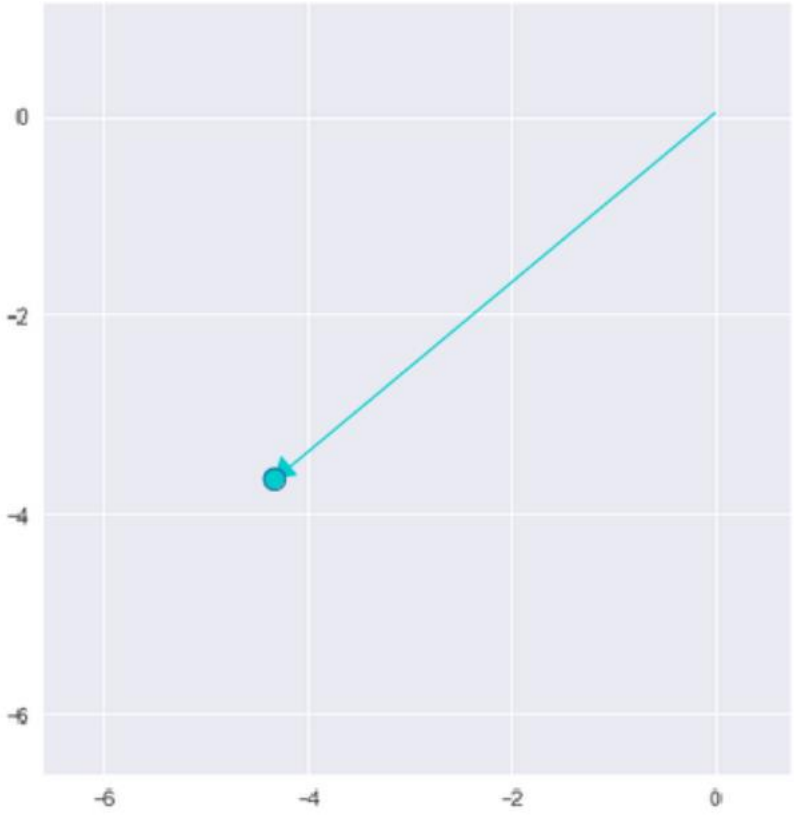
Reconstruction loss

KL divergence between the encoder's distribution and *p(z)* which is a standard normal zero mean and unit variance distribution

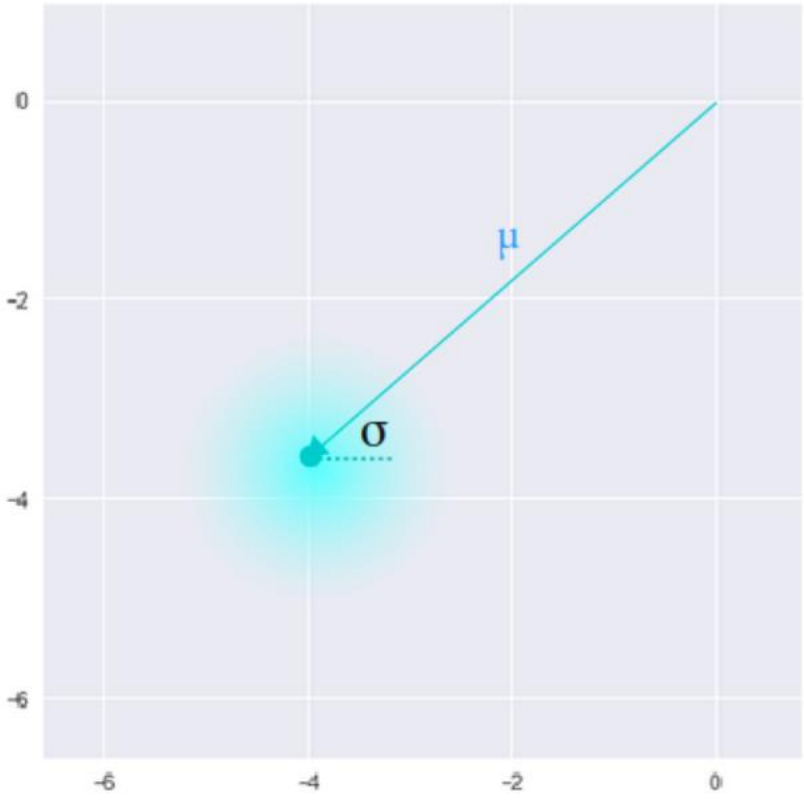


Stochastically generating encoding vectors

For example: For each digit of MNIST you learn the μ and σ vectors. Then to generate a new digit, you sample from the 30 (in the example previous slide) Normal distributions and send those values to the decoder.

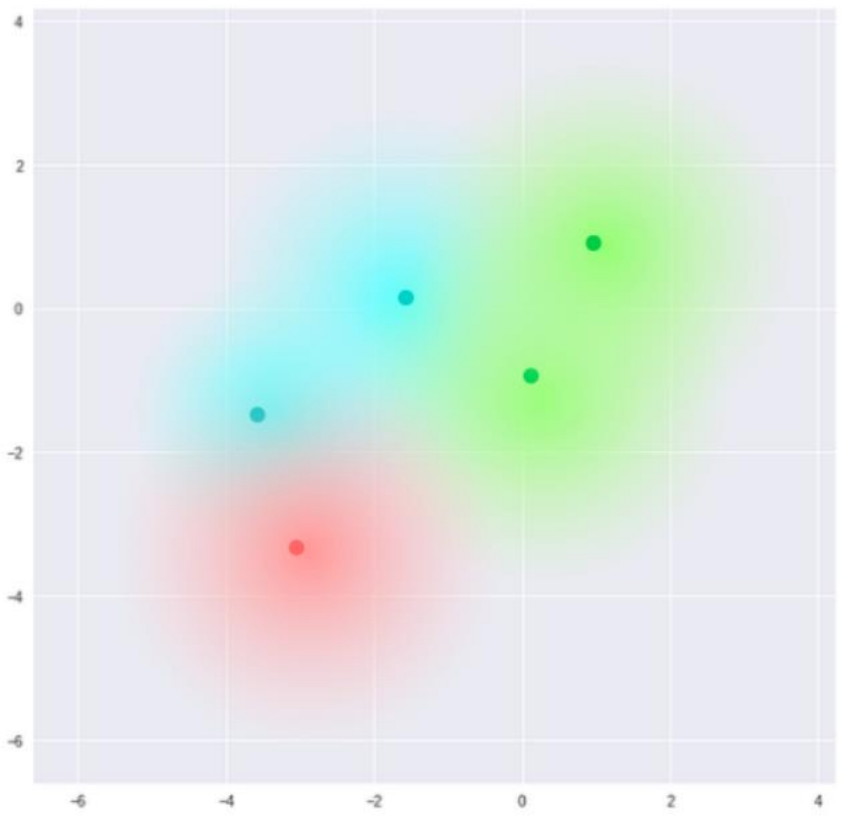


Standard Autoencoder
(direct encoding coordinates)

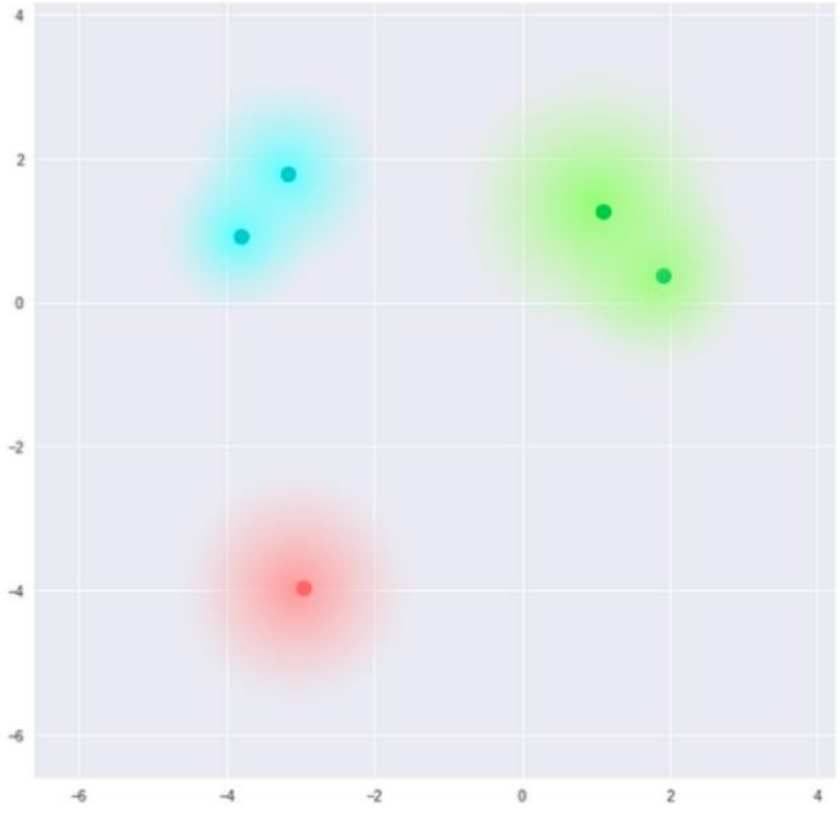


Variational Autoencoder
(μ and σ initialize a probability distribution)

Unfortunately, the VAE will learn the encoding on the right which will not allow samples to be created that “lie in between”



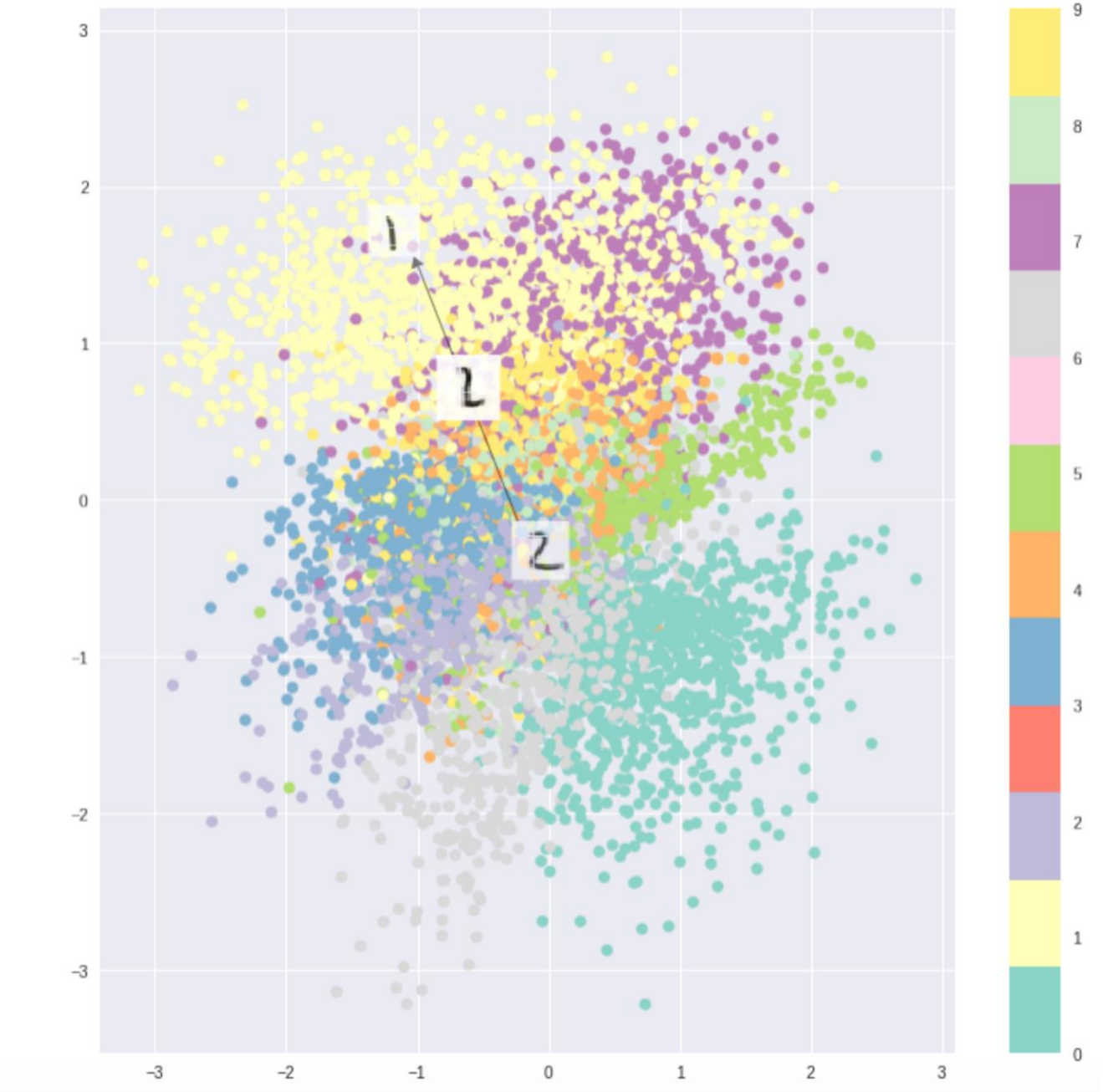
What we require



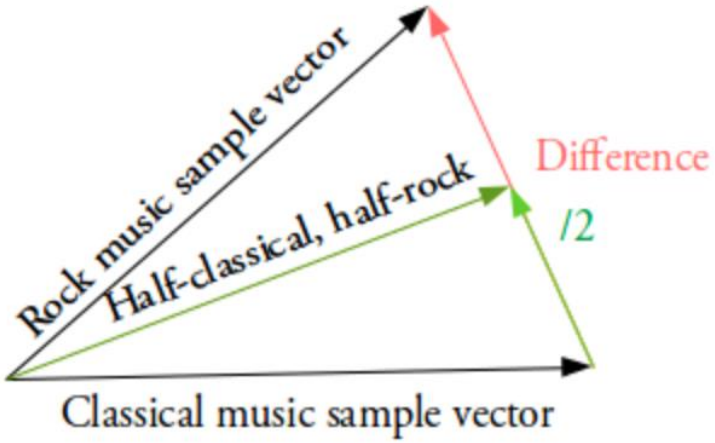
What we may inadvertently end up with

To force VAE to learn encodings that are close to one another, we add a KL loss term. The KL loss is computed w.r.t. the zero mean standard normal distribution.

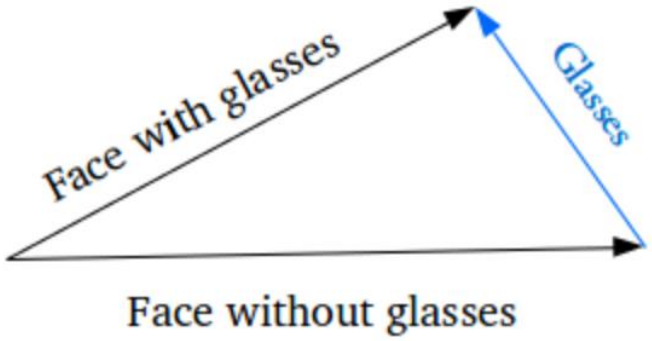
For instance, we see that the encoding for 2 and 1 are close and we can create outputs that are in between.



We can now interpolate between samples or add features to samples etc.



Interpolating between samples



Adding new features to samples

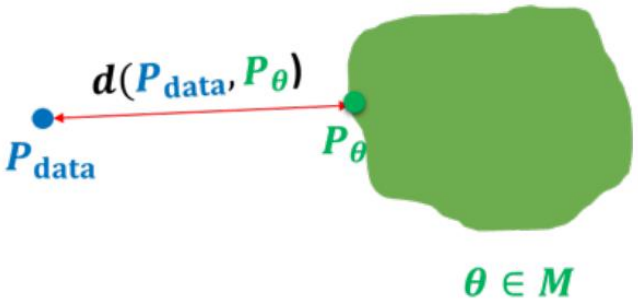
Generating celebrity-lookalike photos

Recap

- Model families we have looked at:
  - Autoregressive models: $p_\theta(X) = \prod_{i=1}^{n} p_\theta(x_i | X_{<i})$
  - Variational autoencoders: $p_\theta(X) = \int p_\theta(X, Z) dZ$

All these models are based on maximizing maximum likelihood or approximations. Is this the best approach?

$\mathbf{x}_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Why Log Likelihood may not yield good samples

- Case 1: We memorize the training set and generate images from that set. The log likelihood is maximized but the samples are uninteresting
- Case 2: Our model is almost entirely noise and therefore so are the generated samples. This also has high log likelihood. Because,

$$p_\theta(X) = 0.01 p_{data}(X) + 0.99 p_{noise}(X)$$

Assume this is our model

Taking logs,

$$\log\ p_\theta(X) = \log[\,0.01 p_{data}(X) + 0.99 p_{noise}(X)] \geq \log 0.01 p_{data}(X) = \log\ p_{data}(X) - \log 100$$

$$E_{p_{data}}[\log p_\theta(X)] \geq E_{p_{data}}[\log p_{data}(X)] - \log 100$$

Lower bound on expected value of log likelihood

$$E_{p_{data}}[\log p_{data}(X)] \geq E_{p_{data}}[\log p_\theta(X)]$$

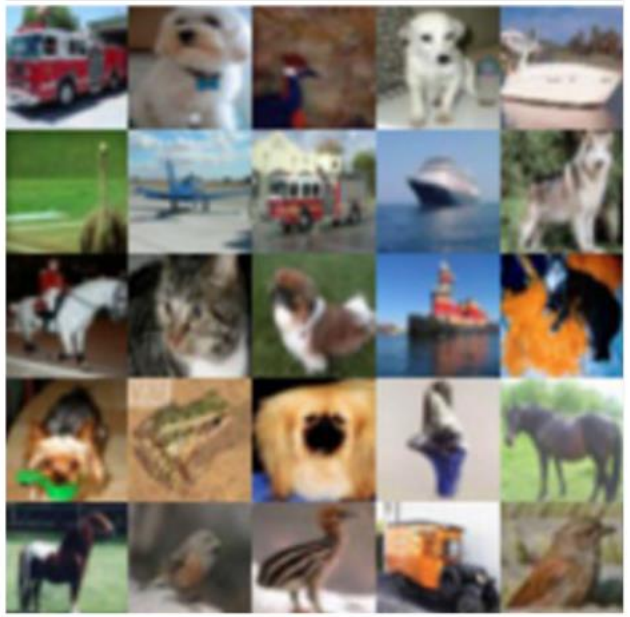
Upper bound on expected value of log likelihood

As we increase the dimensionality of X the absolute value of $\log p_{data}(X)$ increases proportionally but $\log 100$ remains constant. Therefore, $E_{p_{data}}[\log p_{data}(X)] \approx E_{p_{data}}[\log p_\theta(X)]$

Alternate Approach: Compare Distributions using Samples

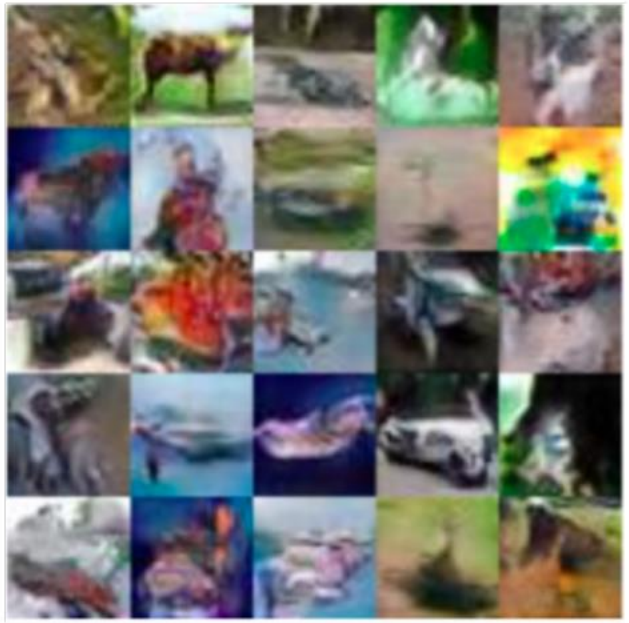
Given a <u>finite</u> set of samples from the two distributions S_1 and S_2 can we tell if they are from the same distribution? In other words, $P = Q$?

Idea: Compute mean and variance of each of the two samples and see if they are close **in some distance measure**.



$S_1 = \{\mathbf{x} \sim P\}$

vs.



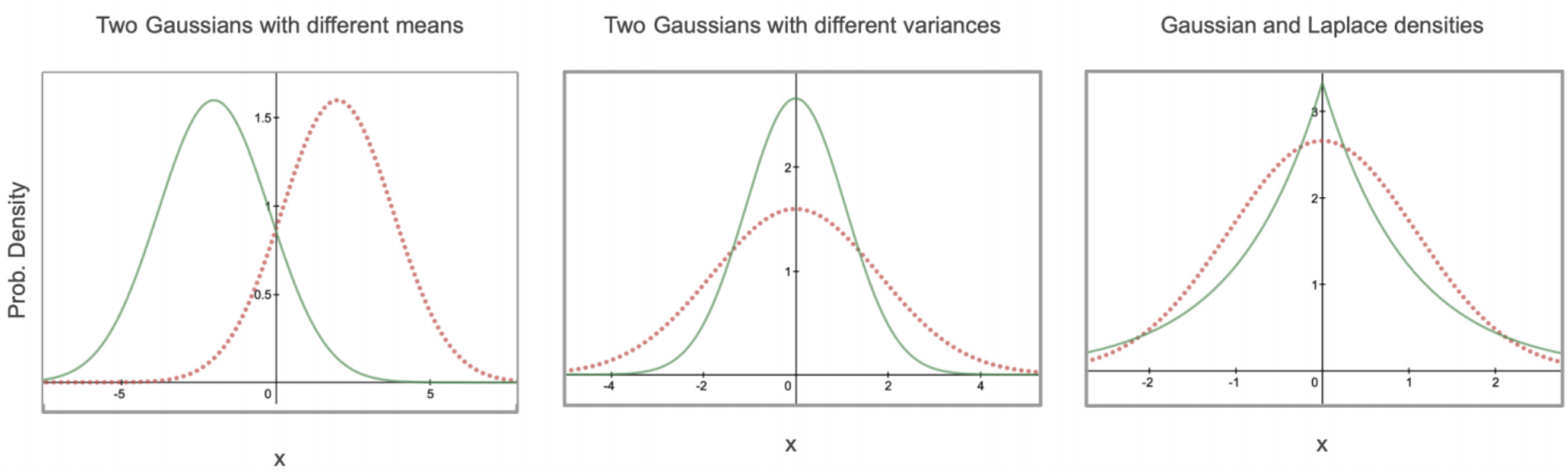
$S_2 = \{\mathbf{x} \sim Q\}$

Note that this is likelihood-free since it does not determine densities but relies on comparing test statistics

Generative Modeling

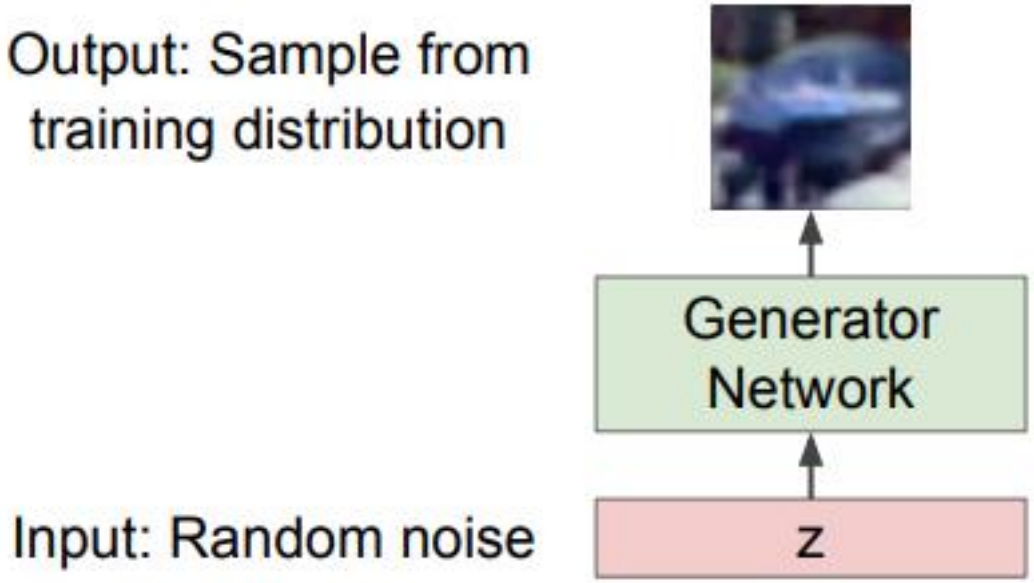
- We have direct access to $S_1 = D = \{X \sim p_{data}\}$ since we have the training set
- We have a model distribution (some learned NN) so $S_2 = \{X \sim p_\theta\}$
- **Approach:** Train the generative model to minimize the two-sample test objective between S_1 and S_2

It is challenging to measure “distance”



Generative Adversarial Networks (GANs)

- Unlike autoencoders, we give up on explicitly modeling the probability density. Rather we just want to sample the distribution.
  - How can you do this?
- Approach: Sample a simple distribution (noise) then learn transformations to convert this to the training distribution!



Two-Person Zero Sum Game

		Player 2	
		1	2
Player 1	1	2,0	0,2
	2	0,2	2,0

Each player holds up one or two fingers. If both hold up 1 or 2 then player 1 wins $2. Otherwise player 2 wins $2. Both players are rational and greedy.

What is the optimal strategy for each player?

Prisoner’s Dilemma

		Player 2	
		Cooperate (with 1)	Defect
Player 1	Cooperate (with 2)	-1,-1	-3,0
	Defect	0,-3	-2,-2

The cooperative strategy, where both prisoner’s cooperate with each other is best (lowest total cost). But, each prisoner can defect in the hope that they get a 0. If both do this, then both get -3.

Nash Equilibrium

The optimal outcome of a game where neither player has an incentive to deviate from her chosen strategy.

		2: I	2: A
1	I	2,1	0,0
	A	0,0	1,2

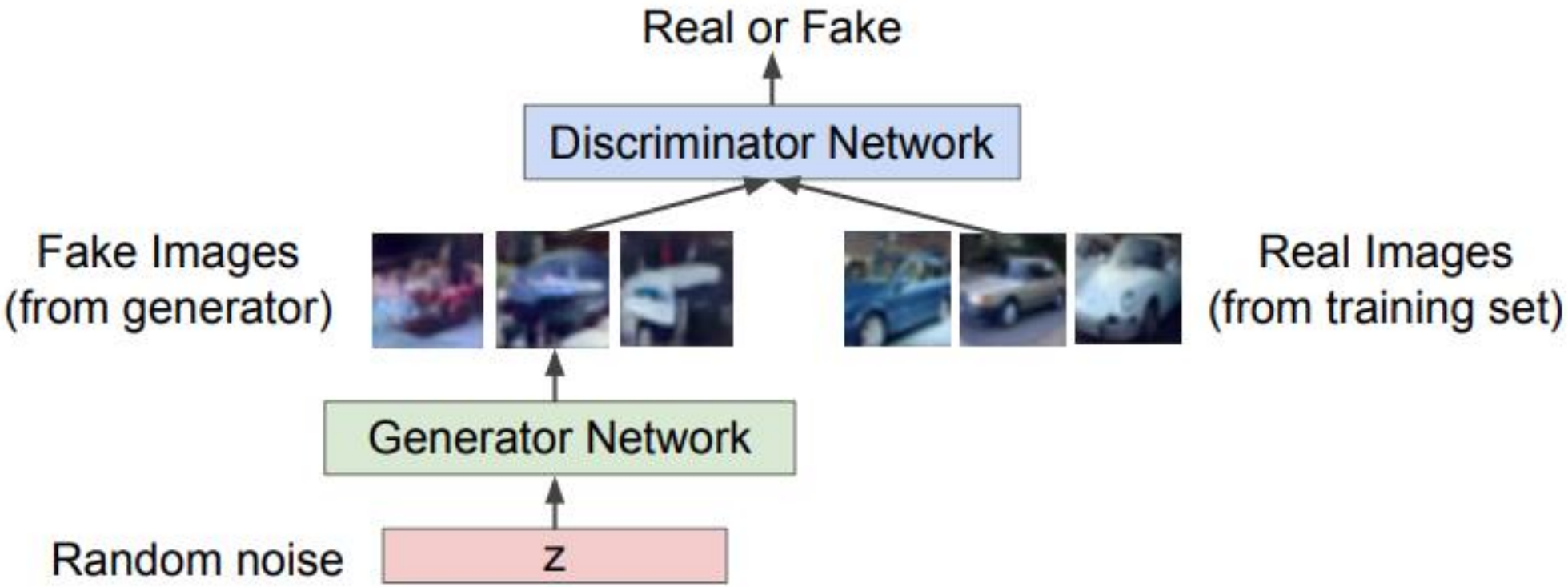
(I,I) neither player can increase payoff by selecting a different choice
(I,A) 1 can increase payoff by selecting A
(A,I) ditto for 2
(A,A) neither can improve payoff by changing strategy
So, (I,I) and (A,A) are Nash equilibria

		2: I	2: A
1	I	2,1	1,2
	A	1,2	2,1

No Nash equilibrium

Train GANs: Two-Player Game

Generator network: try to fool the discriminator by generating real-looking images
Discriminator network: try to distinguish between real and fake images



Discriminator output for real data

Discriminator output for fake data

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

G tries to maximize
D tries to minimize

- Two player minimax game between generator (G) and discriminator (D)
- (D) tries to maximize the log-likelihood for the binary classification problem [data: real (1), generated: fake (0)]
- (G) tries to minimize the log-probability of its samples being classified as “fake” by the discriminator (D)

Two-Player Game

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

2. **Instead: Gradient ascent** on generator, different objective

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$

Alternative

Algorithm

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

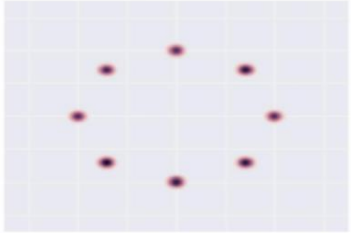
- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

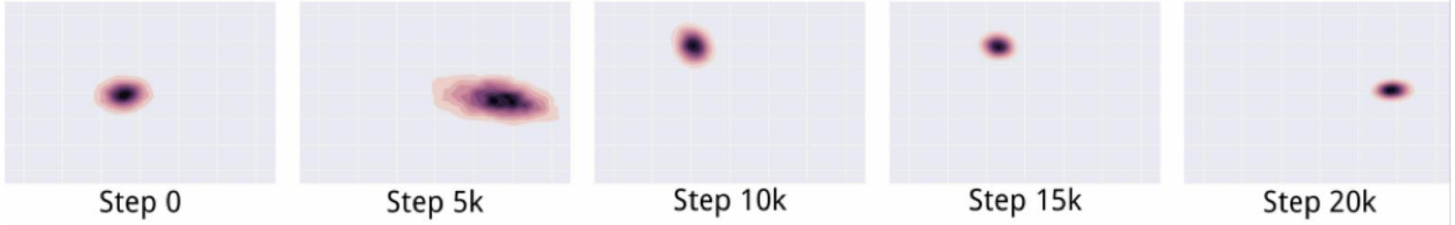
end for

Challenges to Training GANs

- No clear stopping criteria unlike MLE
- Generator and Discriminator oscillate during training
- Mode collapse is a big problem



- True distribution is a mixture of Gaussians



Source: Metz et al., 2017

- The generator distribution keeps oscillating between different modes

Mode Collapse and Failure to Converge

Mode Collapse:

- You want the GAN to produce a wide variety of outputs – e.g., different faces each time
- However, if the generator produces an especially good output, it may learn to produce only that output
- If the generator starts producing only a small set of outputs, the discriminator's best strategy is to reject them all. In the next iteration (after SGD) if the discriminator gets stuck in a local minima, it is easy for the generator to find the most plausible outputs for the current discriminator
- Basically, each iteration of the generator over-optimizes for a particular discriminator and the discriminator never manages to learn its way out of this. As a result, the generator rotates through a small set of output types.

Fixes:

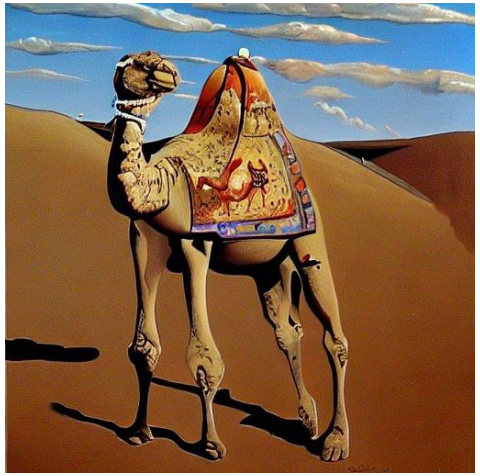
- Techniques to force the generator to broaden its scope by preventing it from optimizing to a particular discriminator

Convergence:

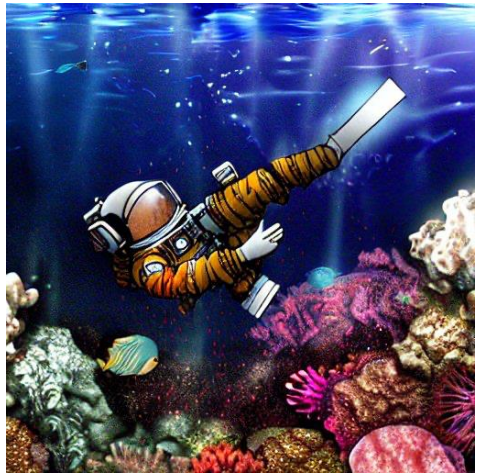
- Add noise to discriminator inputs or penalizing discriminator weights – essentially various forms of regularization

Stable Diffusion

Woman riding a horse



Painting of a camel inspired by Dali



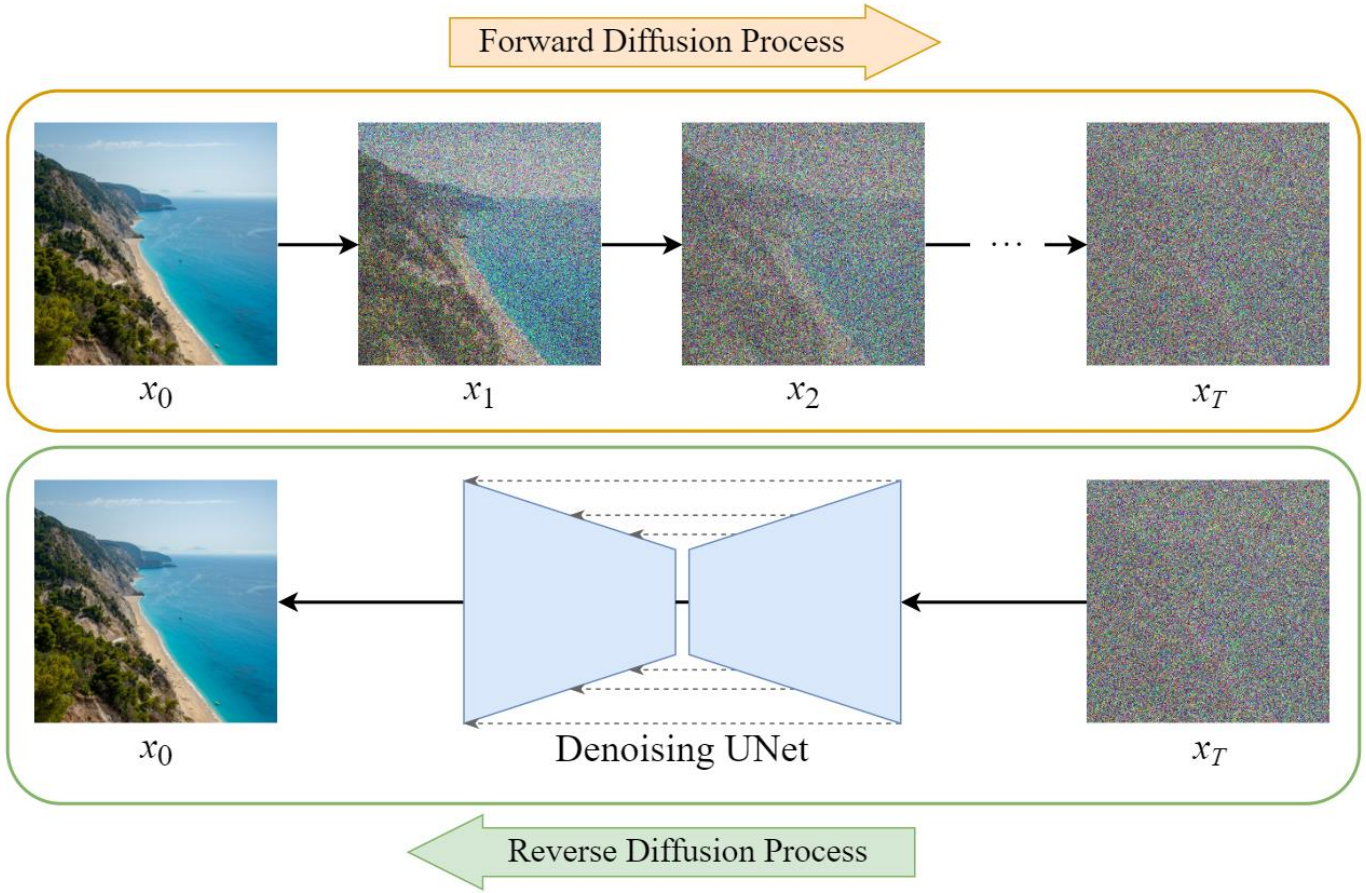
Astronaut underwater

Rock climber

Painting of a field inspired by Matisse

Stop sign in space

Diffusion Model

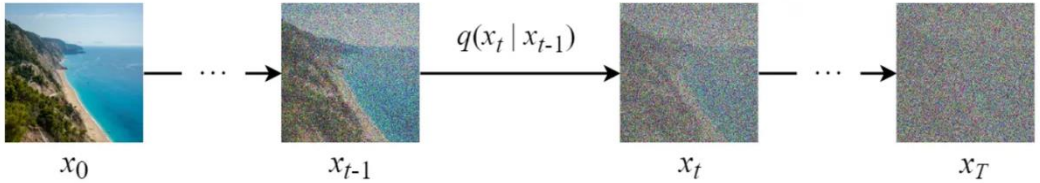


Training consists of two parts:

1. Forward diffusion → add noise to image
2. Reverse diffusion → remove noise

Forward Diffusion

- Noise is added *gradually* to an image (in multiple time steps). If x_0 is the original image then $x_1, x_2, \ldots, x_T$ denote a sequence of increasingly noisy images. Note that as $T \to \infty$ the image is pure noise sampled from a Gaussian distribution



$$q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

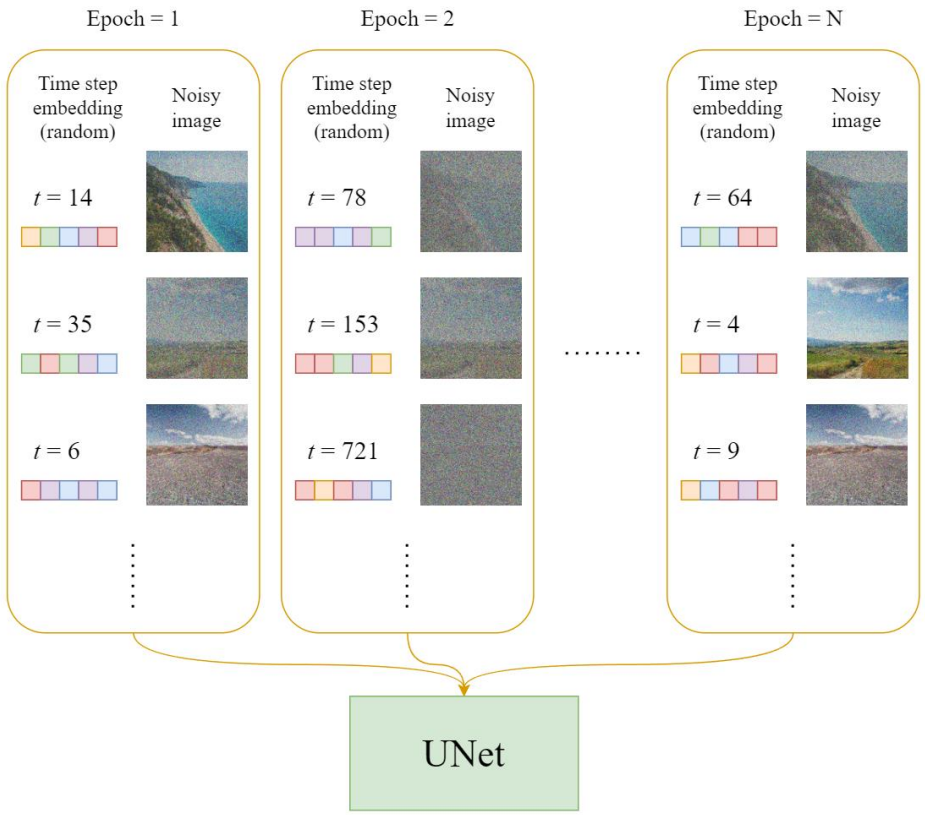
β_0 small number,....., β_T large number (variance); I is the identity matrix

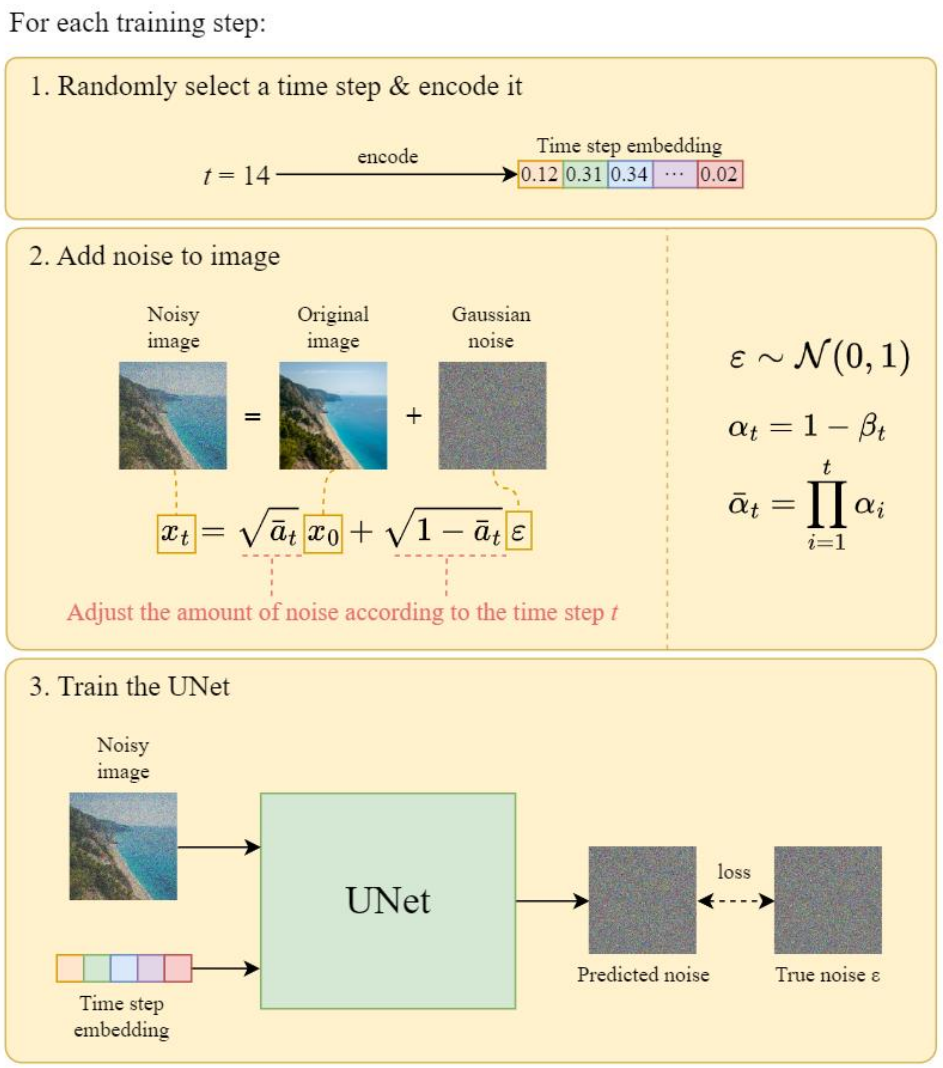
Note that if $\varepsilon \sim N(0,1), x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_{t-1}$

At each step only a little noise is added because it is impossible to reverse the process otherwise

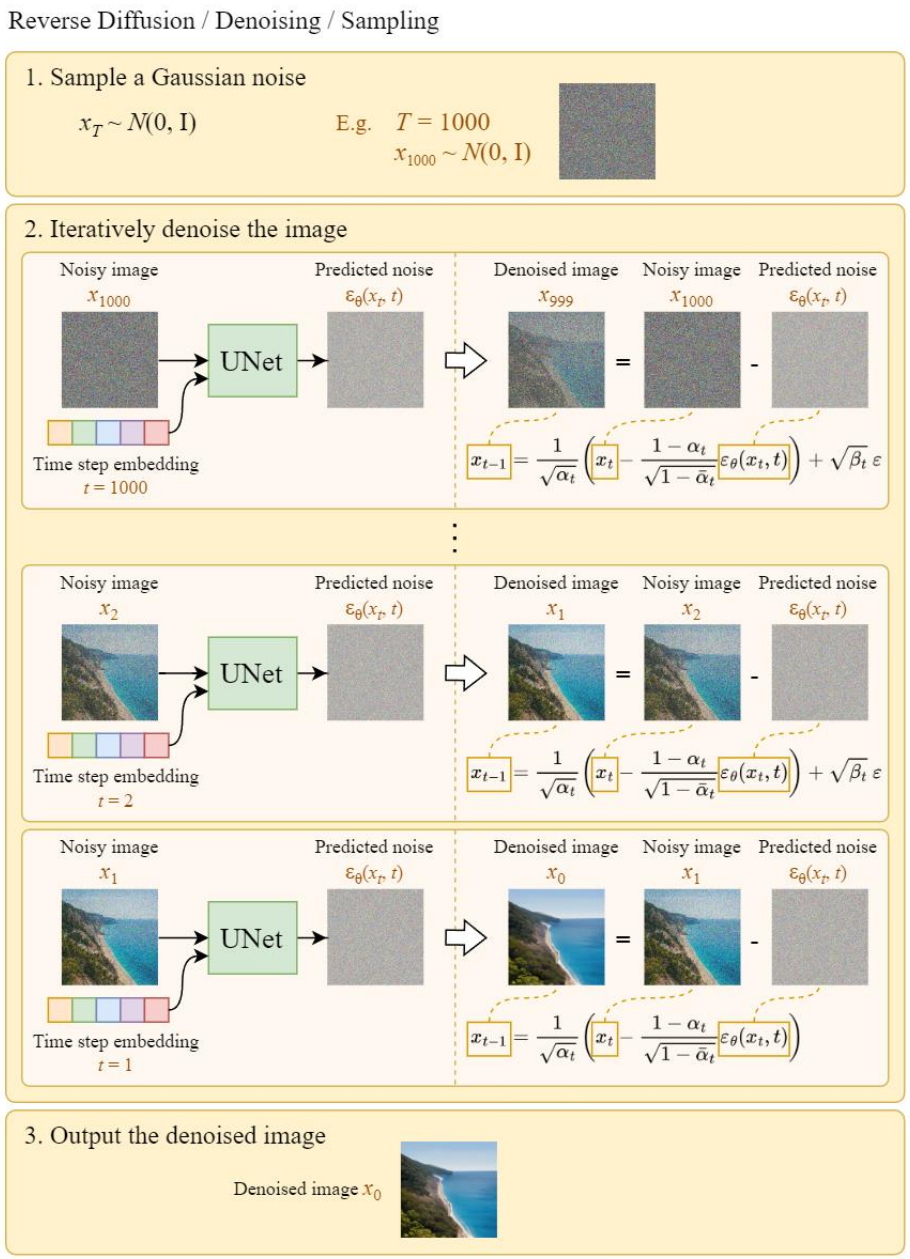
Reverse Diffusion

- Unlike the forward diffusion process, we cannot use $q(x_{t-1}|x_t)$ to reverse the noise since it is uncomputable so we need to train a neural network $p_\theta(x_{t-1}|x_t)$ to approximate $q(x_{t-1}|x_t)$
- At each epoch:
  - A random time step *t* is selected for each training image
  - Apply Gaussian noise corresponding to *t* time steps to the image
  - Convert the time steps *t* to an embedding (vector)



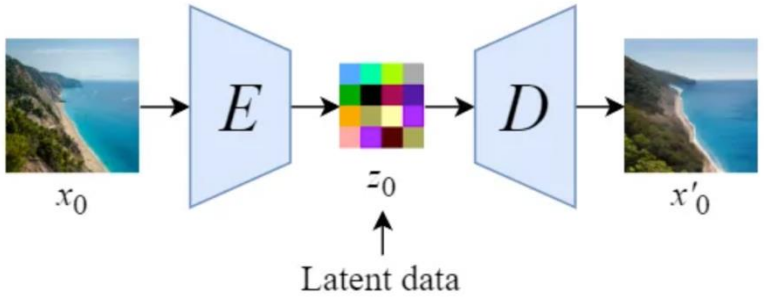


Sampling (Generating an Image)

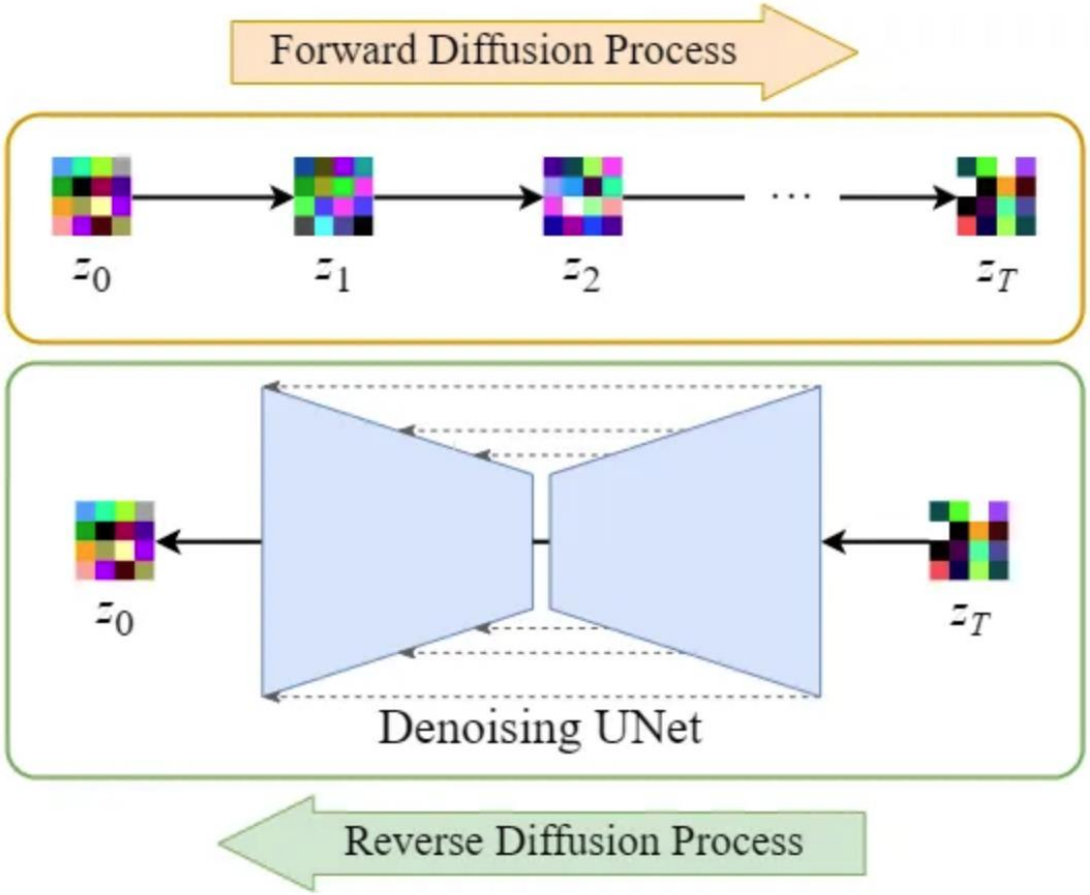


Stable Diffusion or Latent Diffusion Model

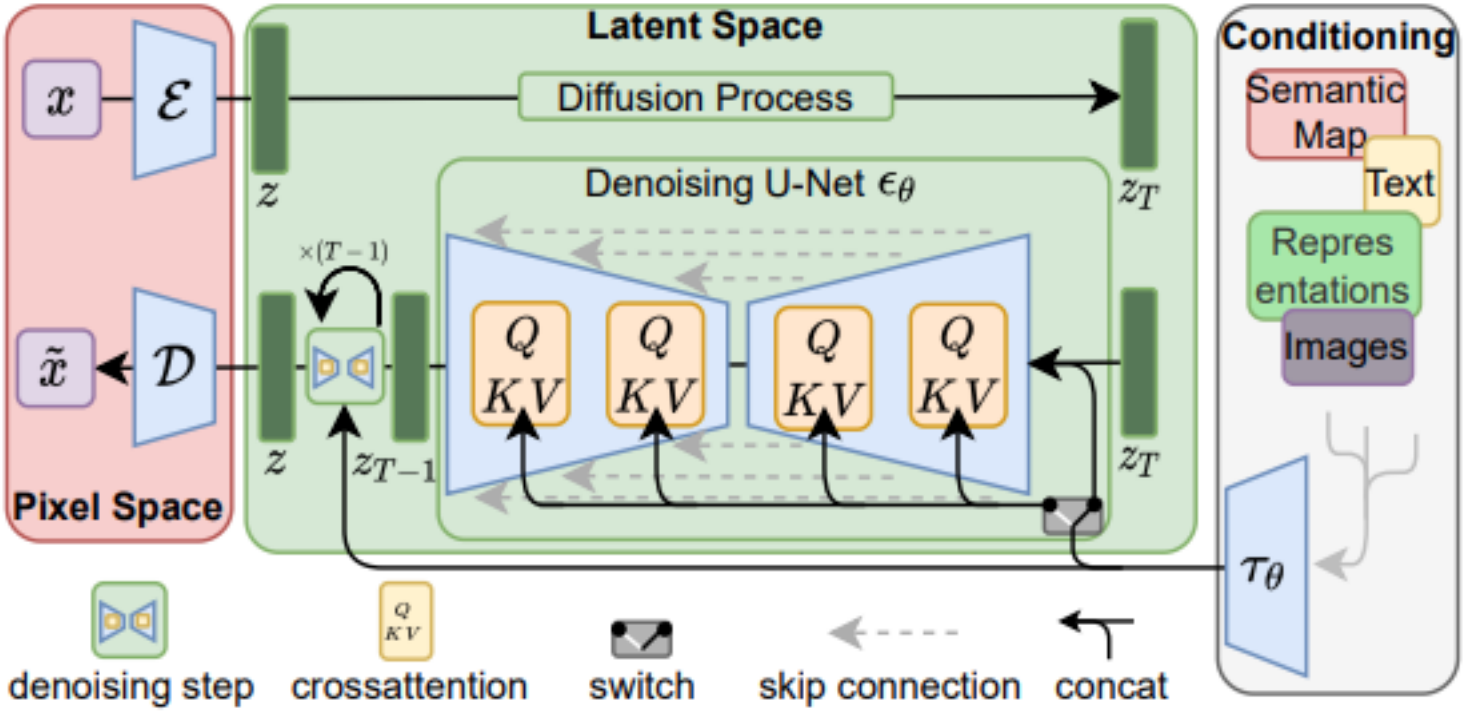
First train an autoencoder to compress the image into latent space (this helps with speed problems)



Then apply the diffusion model to the latent space representation



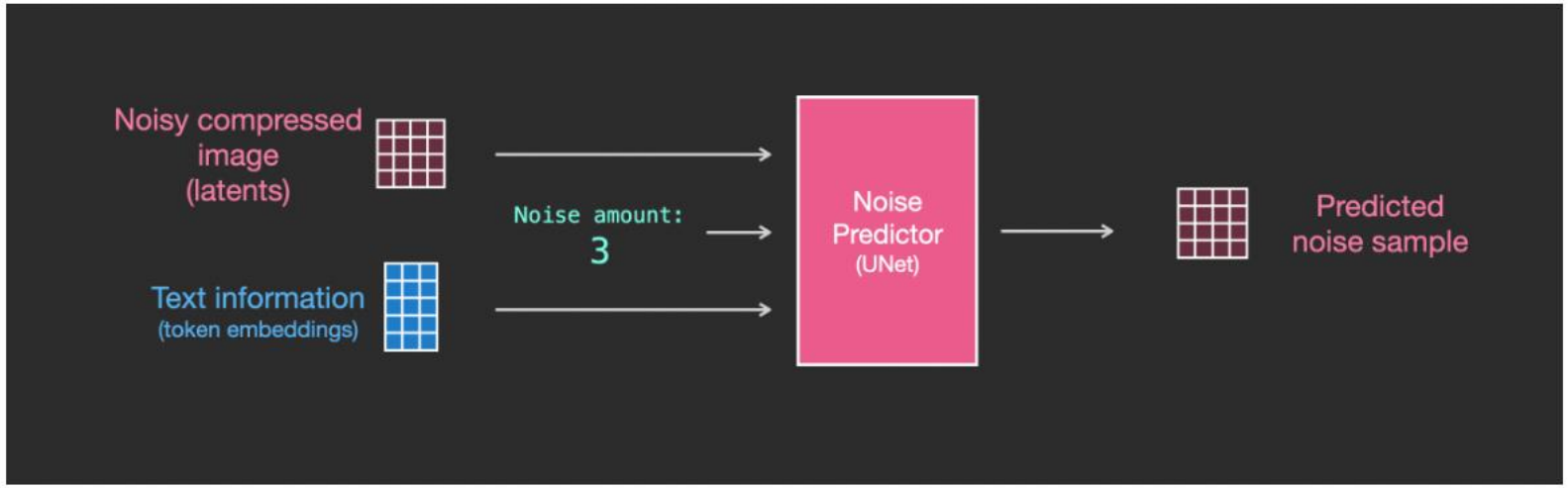
Conditioning (Generating Images from Text or Style Transfer etc.)



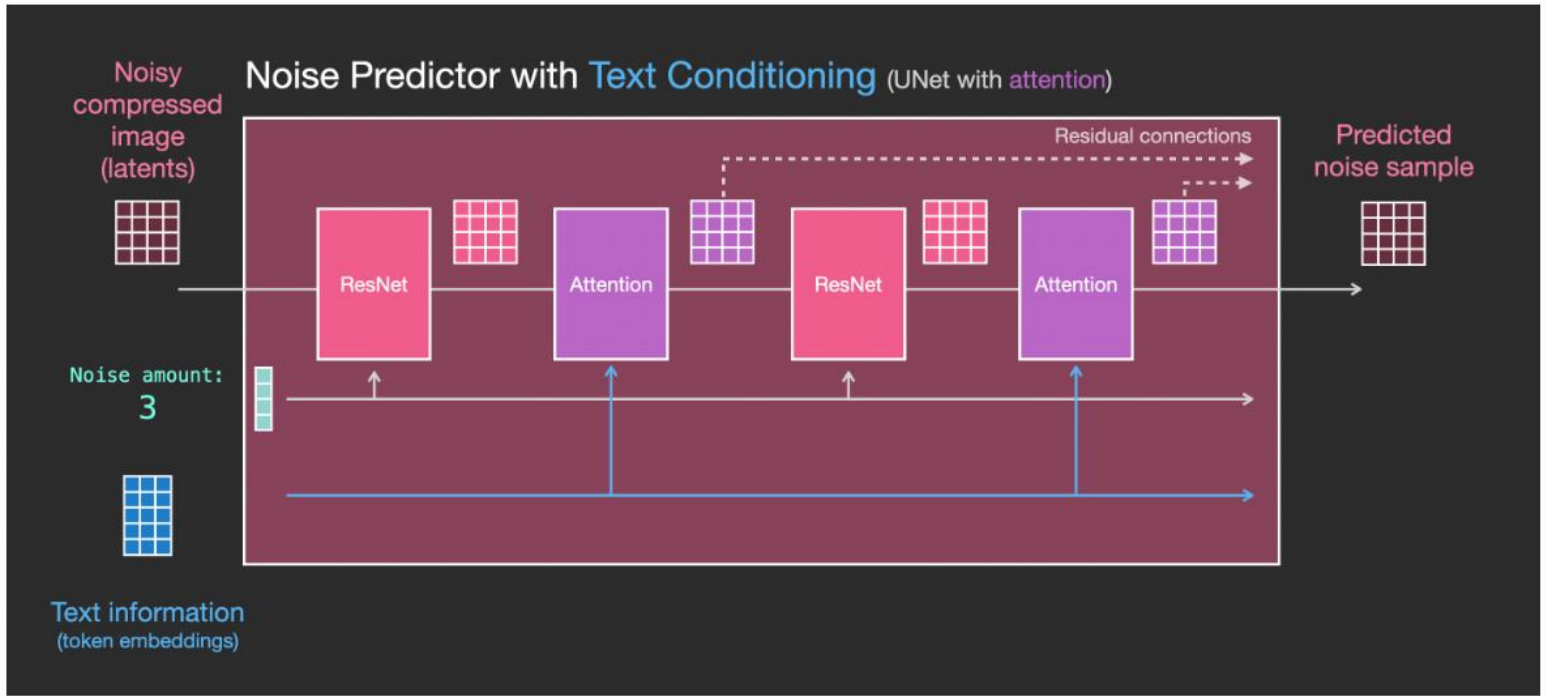
Different possible conditionings

Add a cross-attention mechanism to the basic UNET architecture – cross-attention was implemented in the original Transformer paper where encoder output is fed to the inputs of each fully-connected layer in the decoder.

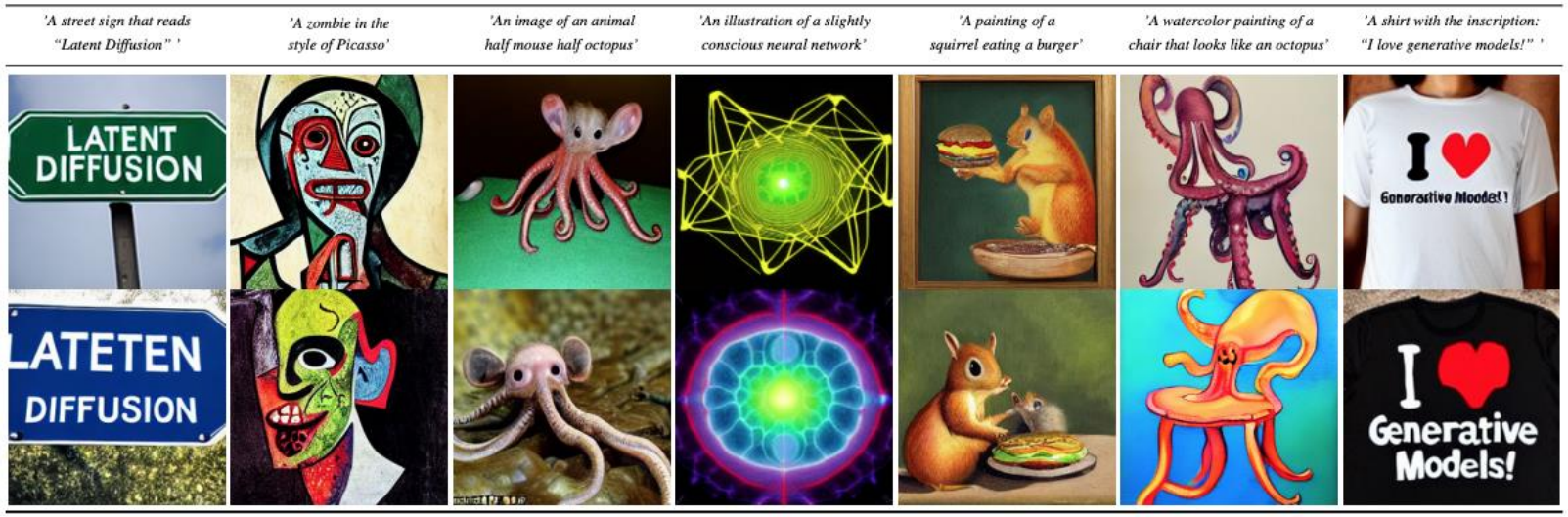
Text Based Image Generation



Basically the text embedding is added between each ResNet block via an attention layer



Text-to-Image Synthesis on LAION. 1.45B Model.



Super-resolution

Figure 9. A *LDM* trained on 256^2 resolution can generalize to larger resolution (here: 512×1024) for spatially conditioned tasks such as semantic synthesis of landscape images. See Sec. 4.3.2.